TRUGS: A Formalized Language Specification for Executable Graph Communication Between Humans and Large Language Models

William E. Leigh III
TRUGS LLC
admin@trugs.ai
ORCID: 0009-0009-1201-3523

Abstract

Two graph traditions run through computing. *Semantic* graphs—from semantic networks through RDF, OWL, and modern knowledge graphs—carry meaning but do not execute: they store assertions for a separate program to read. *Executable* graphs—from Petri nets through dataflow architectures, statecharts, and the computation DAGs of modern ML systems—execute but do not carry meaning: their nodes are opaque operations. Communication between humans and large language models (LLMs) sits in the gap between the two: natural language is inherently ambiguous, structured formats are precise but not human-writable, and the extraction pipelines that translate between them introduce error at each stage. We propose TRUGS (Traceable Recursive Universal Graph Specification), a formalized language that draws on both lineages: every valid sentence compiles to a directed graph, every graph decompiles to the same sentence, and the graph is executable—typed nodes carry operations, topology defines execution order, and constraint edges encode governance. Bidirectional controlled natural languages exist (Sydney OWL Syntax; ACE), but they compile to formalism-specific ontology logic; TRUGS compiles to a general-purpose executable graph. TRUGS combines capabilities that have previously appeared separately in controlled-language executable specifications, semantic web-service models, model-based systems engineering, and executable knowledge graphs.

TRUGS draws its 233-word vocabulary from computation (67%) and law (20%)—two domains with centuries of precedent in making English words formally precise—with the remaining 31 words shared scaffolding (quantifiers and the SI level prefixes that mark hierarchy). The vocabulary is organized into 9 parts of speech governed by a BNF grammar and a composition type system; the CORE-16 contract (9 structural rules, always enforced, plus 7 compositional rules, enforced when a graph declares a CORE vocabulary capability) validates every graph. We demonstrate empirically, on the 30 worked examples of the specification's example file, that the compilation is bidirectional and lossless: the released tool accepts 29 of them and decompiles 29 compiled graphs back to their source text (the example file marks the unaccepted form as reserved for a grammar production the parser does not yet implement). An open "sugar" pattern (`'word`) admits unlimited human-readability tokens that compile to nothing.

The system is language-agnostic by design: legal concepts such as obligation (*shall*/应当/*deberá*) and computational operations such as *filter* and *validate* exist independently across major human languages. Loading a different vocabulary definition—itself a TRUG—switches the surface language while preserving the grammar and graph structure.

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

We release the complete specification, the language's self-describing definition (itself a TRUG), and worked examples under the Apache 2.0 license, together with a published reference toolchain: `trugs-tools` (the `trug` CLI—validation, TRL compilation, corpus audit) and `trugs-folder` (`trug-a-folder`, filesystem↔graph cartography).

1 Introduction

When a developer instructs an LLM to "make sure the API handles errors gracefully," the instruction is syntactically valid English, yet it admits an unbounded number of interpretations. In ordinary practice, neither party has a mechanical way to verify which interpretation the model adopted. In multi-agent settings, where LLMs coordinate through natural language exchanges, this interpretive uncertainty compounds with each step of the interaction.

We argue that this is fundamentally a **language** problem, not a tooling problem. LLM practice has not adopted a shared language that is simultaneously readable by humans, precise enough for machine execution, and formal enough for mechanical validation; the controlled languages that come closest (Section 3.2) target logic programs, ontologies, or model simulation rather than a general-purpose executable graph.

Three specific manifestations of this problem drive current research:

1. **Natural language is inherently ambiguous.** Human language evolved for social communication, where shared context, tone, and experience disambiguate meaning. When repurposed for machine instruction, these cues are absent. A sentence such as "handle the data" admits numerous valid interpretations, and, in ordinary practice, no verification step confirms which one an LLM selected.
2. **Structured formats sacrifice readability for precision.** JSON-LD, RDF, OWL, and property graphs provide machine-processable representations, but they are not human-writable at scale. This gap motivated the field of knowledge graph construction [Bian, 2025], which is dedicated to translating natural language into structured form—a process that introduces error at each stage of the pipeline.
3. **Current LLM communication protocols leave the semantics of terms undefined.** Protocols such as MCP [Anthropic, 2025] standardize the mechanics of tool invocation (request/response patterns, tool schemas) but leave the semantics of individual terms undefined. Two LLMs using MCP can invoke the same tool without any guarantee that they share the same operational definition of "filter" or "validate." The agent-orchestration frameworks that emerged alongside these protocols—LangGraph, AutoGen, CrewAI [LangChain AI, 2024, Wu et al., 2023, Moura, 2023]—coordinate LLMs *through* graphs, but those graphs define control flow, not meaning: the nodes remain opaque functions.

We observe that two established professional domains have already addressed this problem, independently and over centuries. In **computation**, English words such as `filter`, `sort`, and `return` carry deterministic meanings enforced by compilers. SQL, designed in 1974 as a "Structured English Query Language" [Chamberlin and Boyce, 1974], demonstrated that English words with formal semantics could serve as a universal interface; it remains the most widely adopted query language today [Chamberlin, 2012]. In **law**, English words such as *shall*, *may*, and *notwithstanding* carry formally defined scope enforced by courts. The formal study of obligation, permission, and prohibition—deontic logic—dates to von Wright [von Wright, 1951], and the IETF later codified computational equivalents in RFC 2119 [Bradner, 1997], defining MUST, SHALL, MAY, and SHOULD as formal requirement levels. In both cases, ordinary English words were constrained into precise, executable vocabulary without inventing new terminology.

TRUGS applies the same principle to human-LLM communication. We curated 233 words from computation and law, organized them into 9 parts of speech with a formal grammar, and defined composition rules that govern valid combinations. The resulting language compiles bidirectionally to a directed graph. To our knowledge, prior systems do not combine a losslessly bidirectional controlled natural language with a domain-general executable semantic graph in which nodes encode operations, topology encodes control flow, and typed edges encode deontic governance, specifically

for human–LLM and inter-agent communication. Section 3.2 cites the closest earlier systems and states what each lacks. The central insight is that the required vocabulary and its formal semantics already exist in established professional traditions; the contribution is their assembly into a unified, compilable language.

1.1 Contributions

1. A **graph specification** (TRUGS CORE) with 10 semantic primitive classes, 7 structural boundaries, and the CORE-16 validation contract (9 structural + 7 compositional rules), providing a universal foundation for structured information representation.
2. A **formalized English language** of 233 words in 9 parts of speech with a BNF grammar, composition rules, and a bidirectional compilation guarantee: sentence $\leftrightarrow$ graph, losslessly.
3. The **opening TRUG** pattern, where the language definition itself is a TRUG—enabling domain extensibility and self-description.
4. A published **reference toolchain**: `trugs-tools` (the `trug` CLI—validator, TRL compiler, corpus audit) and `trugs-folder` (`trug-a-folder`, filesystem cartography), both installable from PyPI.
5. The complete specification, the self-describing language definition, and examples released under Apache 2.0 at `https://github.com/TRUGS-LLC/TRUGS`.

2 From Semantic Graphs to Executable Language

Before presenting the formal specification, we introduce the core concepts through a concrete example.

2.1 Semantic Graphs

A semantic graph represents knowledge as typed nodes (things) connected by labeled edges (relationships). Unlike flat data formats, a semantic graph encodes not just what things exist but how they relate to one another. The nodes carry properties; the edges carry meaning. A node of type `PARTY` connected by a `GOVERNS` edge to a node of type `RESOURCE` does not merely associate two records—it asserts a specific relationship (authority) between two specific kinds of entities (an actor and an asset).

Semantic graphs have a long history in knowledge representation, reaching from Quillian's semantic memory networks [Quillian, 1968] through the Semantic Web program [Berners-Lee et al., 2001] and ontology engineering [Gruber, 1993]; standards such as RDF [W3C, 2014] and OWL [W3C, 2012] formalize their structure. However, conventional semantic graphs are *representational*: they store assertions about the world. They do not, by themselves, define a sequence of operations or enforce constraints on who may do what to whom.

2.2 Executable Graphs

The executable-graph tradition is as old as the semantic one, and for most of its history it evolved in parallel with it; Section 3.2 discusses the systems that bridge the two. Petri nets [Petri, 1962] made the graph itself the computation: execution is the firing of transitions. Karp and Miller formalized parallel computation as graph models [Karp and Miller, 1966]; Dennis's dataflow architectures made "the graph is the program" a hardware thesis [Dennis and Misunas, 1975]; Hewitt's Actor model recast computation as a graph of message-passing agents [Hewitt et al., 1973]; and Harel's statecharts demonstrated that a graph formalism can be simultaneously human-readable and executable [Harel, 1987]. The lineage runs unbroken into the present: the computation DAGs of TensorFlow [Abadi et al., 2016] and Spark [Zaharia et al., 2012] are dataflow graphs at datacenter scale.

What these graphs lack is precisely what the semantic lineage has: meaning. Their nodes are opaque operations—the graph defines *how* computation flows, not *what* it means. Conversely, semantic graphs carry meaning that only a separate program can act on. Partial bridges exist: OWL reasoning engines execute inference over semantic graphs, and ExeKG translates knowledge graphs into executable analytics scripts [Zheng et al., 2022]—but in each case the graph and the execution remain separate artifacts, one carrying the meaning and the other performing the work.

An **executable graph** in the TRUGS sense addresses this gap: it extends a semantic graph into a computational structure—one that defines operations, not merely assertions. Three properties distinguish an executable graph from a conventional semantic graph:

1. **Nodes carry executable content.** Each node is simultaneously data (describable, queryable) and an instruction (performable by any agent that traverses the graph). Writing a node into the graph defines an instruction and makes it available for execution, with no compilation step or deployment boundary.
2. **Topology defines execution.** The topology of nodes and edges defines a sequence of computational operations. A `THEN` edge means "execute this after that." An `OR` edge means "execute this if that fails." Edges are structural constraints—they define the only available traversal paths. The graph does not advise a sequence; it enforces one.
3. **Constraint edges encode governance.** Edges such as `SHALL` and `MAY` encode obligations and permissions that govern which actors may perform which operations. These constraints are structural elements validated mechanically by the same rules that validate the graph's topology.

The executable graph is **agent-agnostic**: any computational agent capable of reading nodes, following edges, and executing content can traverse it. Different agents executing the same graph may produce different results—but all results are valid with respect to the topology. The graph is the program; the agent is the execution engine.

This is the foundation on which TRUGS is built.

2.3 TRUGS: A Concrete Example

A TRUG (Traceable Recursive Universal Graph) is an executable graph serialized as a JSON file with a fixed structure: 7 required node fields, 3 required edge fields, and a set of validation rules. A computational agent—whether a large language model, a rule engine, or a human reader—can traverse a TRUG, following edges to execute the operations encoded in its topology. Consider the following TRUGS Language sentence:

```
PARTY system SHALL FILTER ALL ACTIVE RECORD
  THEN WRITE RESULT TO ENDPOINT output.
```

This sentence compiles to a graph with operation nodes for `FILTER` and `WRITE`, connected by `THEN`.

The full compiler output is in Appendix E.1.

The released decompiler reconstructs the sentence from this graph. The graph includes required structural fields as well as the nodes, edges and noun-phrase properties that encode the source. The sentence and the graph are two renderings of the same artifact.

Several features of this example are worth noting:

- `PARTY system` declares a computational agent. `SHALL` assigns an obligation to that agent via a constraint edge—governance encoded as graph structure, not as prose commentary.
- `FILTER` and `WRITE` are operation nodes: they are simultaneously data (describable, queryable) and executable content (a computational agent performs them during traversal).
- `THEN` is a topological edge encoding execution order. A computational agent traversing this graph must execute `WRITE` after `FILTER`—not because a text instruction says so, but because the topology permits no other path.
- `ALL ACTIVE RECORD` combines a selector (`ALL`), a modifier (`ACTIVE`), and an entity (`RECORD`)—three distinct graph elements composed according to formal rules and validated by the composition type system.

The remainder of this paper formalizes this example: Section 3 surveys related approaches, Section 4 specifies the graph structure, Section 5 defines the language, Section 6 presents additional worked examples, and Sections 7–10 describe the compilation mechanism, evaluation, and discussion.

3 Related Work

The idea of making natural language more precise for machines is not new. Four research traditions have addressed aspects of this problem; TRUGS draws on all four while making different design choices. Section 3.2 then takes up the five earlier systems that come closest to TRUGS and states what each lacks.

3.1 Controlled Natural Languages

Controlled Natural Languages (CNLs) restrict natural language to a subset that can be unambiguously parsed. Kuhn's comprehensive survey [Kuhn, 2014] catalogues over 100 CNLs developed since 1930, forming "a single scattered cloud filling the conceptual space between natural languages and formal languages." Attempto Controlled English (ACE) [Fuchs et al., 2008] is the most prominent, translating a subset of English into first-order logic. ACE and similar systems (CPL [Clark et al., 2005], PENG [Schwitter, 2002]) proved that constrained English *can* be made unambiguous.

However, existing CNLs overwhelmingly target **logic**—their compilation target is a theorem prover or knowledge base. This lineage traces to Montague's foundational work on compositional formal semantics for natural language [Montague, 1973], but it inherits the limitation that logical representations are not data structures that can be directly traversed, mutated, or executed by computational agents.

Notably, several CNLs already support bidirectional mapping. Sydney OWL Syntax [Cregan et al., 2007] provides a bidirectional CNL-to-OWL mapping, and ACE can both generate and parse OWL ontologies. TRUGS builds on this tradition but makes three distinct choices: the compilation target is a **general-purpose directed graph** (not OWL ontology logic), the vocabulary is drawn from **computation and law** rather than general English, and the specification includes a **composition type system** that rejects semantically invalid combinations at parse time. These choices make TRUGS executable rather than merely representational.

3.2 The Closest Earlier Systems

Five earlier systems each supply a part of what TRUGS combines; none supplies the combination, and this section states for each the property it lacks. Attempto [Schwitter and Fuchs, 1996, Fuchs et al., 2008] translates specifications written in Attempto Controlled English into discourse representation structures and into Prolog; the resulting knowledge base can be queried in ACE and run for simulation, prototyping, and validation of the specification. Its compiled artifact is a logic program, not a typed graph a reader can traverse as data, and nothing decompiles the Prolog back to the sentence. Schwitter's bi-directional grammar [Schwitter, 2018] specifies an answer set program in controlled natural language and verbalises the program, possibly after revision, in the same controlled language; the generated specification is semantically equivalent to the original, which that work calls semantic round-tripping. The round trip is semantic and the target is again a logic program. TRUGS's round trip is syntactic: the released tool decompiles the compiled graph to the source text in its canonical layout, token for token, and the target is a typed, domain-general graph. OWL-S [Martin et al., 2004] is an OWL ontology for describing Web services as processes with inputs, outputs, preconditions, and effects, so that software agents can discover, invoke, and compose services automatically; it has no controlled-language surface, so a description is written in OWL or generated by a tool, and nothing decompiles it to a sentence. Object-Process Methodology integrates a system's structure and behavior in a single diagram type, supports executable simulation of the model in its OPCAT environment [Yaroker et al., 2013, Dori et al., 2003], and is standardized as ISO 19450; its text form is generated from the diagram rather than authored first, and its vocabulary carries no deontic modalities, whereas TRUGS's `SHALL`, `MAY`, and `SHALL_NOT` are typed edges of the graph. Executable knowledge graphs [Zheng et al., 2022] encode machine-learning pipelines in semantic knowledge graphs built through a GUI and translate them into executable Python scripts; the approach is specific to machine-learning analytics, the graph is not authored as text, and its vocabulary carries no obligations or permissions. Table 1 summarizes these properties.

Table 1: The earlier systems closest to TRUGS, by the properties TRUGS combines. "Round trip" is between the authored artifact and the compiled artifact.

System	Authored as	Round trip	Compiled artifact	Deontic edges
Attempto [Schwitter and Fuchs, 1996]	controlled English	none (one way)	logic program (Prolog)	no
Schwitter 2018 [Schwitter, 2018]	controlled English	semantic	answer set program	no
OWL-S [Martin et al., 2004]	OWL ontology	none	service process model	no
OPM / OPCAT [Yaroker et al., 2013]	diagram	text generated from the diagram	simulated model	no
ExeKG [Zheng et al., 2022]	GUI-built knowledge graph	none (one way)	Python scripts	no
TRUGS	controlled English	syntactic, token for token	typed JSON graph	yes

3.3 Knowledge Graph Construction with LLMs

Recent surveys [Bian, 2025, Pan et al., 2024] document a paradigm shift in knowledge graph construction from rule-based pipelines to LLM-driven frameworks. The dominant approach uses LLMs to **extract** structured knowledge from unstructured text through schema-based or schema-free methods. The field exists precisely because of the representational gap between natural language and structured graphs; bridging this gap requires extraction pipelines, and each pipeline stage introduces potential information loss.

TRUGS addresses this problem from a different angle: rather than improving extraction, it eliminates the need for extraction entirely. If the input language is itself a graph representation, then schema induction, entity extraction, and knowledge fusion become unnecessary—the input is structured by construction. The three persistent challenges identified by Bian [2025]—scalability, expert dependency, and pipeline fragmentation—are artifacts of the translation paradigm rather than inherent properties of the problem.

3.4 Graph Specification Formats

RDF [W3C, 2014], OWL [W3C, 2012], JSON-LD [W3C, 2020], and property graph models [Neo4j, Inc., 2024] provide well-established formal specifications for graph data. These formats offer precision, composability, and mechanical validation, but they are not designed for direct human authoring. In practice, this creates a persistent dependency on intermediate tooling that translates between human-authored input and machine-required structure.

TRUGS provides comparable structural guarantees—typed nodes, labeled edges, hierarchical containment, and validation rules—in a format where the human-authored sentence and the machine-processed graph are the same artifact. No intermediate translation is required.

3.5 LLM Communication Protocols and Agent Frameworks

The Model Context Protocol (MCP) [Anthropic, 2025] standardizes how LLMs invoke external tools. It is an important advance—but it specifies the **mechanics** of communication (request/response patterns, tool schemas) without addressing the **semantics** (what the words mean). Two LLMs using MCP can call the same tool, but they cannot verify that they share the same definition of "filter" or "validate." TRUGS operates at the semantic layer: every word has exactly one meaning, defined in the opening TRUG, shared by all participants in a conversation. MCP and TRUGS are complementary—MCP for mechanics, TRUGS for meaning.

The multi-agent frameworks that emerged in the same period use graphs to *orchestrate* LLMs. LangGraph [LangChain AI, 2024] is a graph-based state machine whose nodes are functions; AutoGen [Wu et al., 2023] structures agents as a conversation topology; CrewAI [Moura, 2023] organizes them as role hierarchies; DSPy [Khattab et al., 2024] compiles declarative language-model pipelines, carrying semantics at module granularity through typed signatures—the closest of the four to a semantic computation graph. In every case, however, the graph carries control flow while the content of each node remains opaque natural language or code: the graph says *when* an agent runs, not *what its instructions mean*. TRUGS is the complement: the graph itself is the semantic artifact. Nodes and edges are typed against a closed vocabulary, composition is validated mechanically, and the same graph is the medium the agents exchange.

3.6 LLM Structured Output

Recent work on constrained decoding [Willard and Louf, 2023, Tam et al., 2025] forces LLMs to produce syntactically valid structured output (JSON, XML) by masking invalid tokens during generation. This approach ensures **format** compliance but not **semantic** compliance. An LLM constrained to produce valid JSON will produce valid JSON—but the JSON may still contain semantically meaningless content. TRUGS addresses both: the language ensures format compliance (the grammar rejects malformed sentences) and semantic compliance (the composition rules reject meaningless combinations like "DATA records SHALL FILTER").

4 TRUGS Specification

Having introduced the executable graph concept informally, we now specify its structure precisely. A TRUG is a JSON-serialized directed graph comprising typed nodes, labeled edges encoding directional relationships, and hierarchical containment. The format is intentionally minimal: it imposes structural invariants that guarantee universal tooling compatibility and agent-agnostic traversal, while leaving property schemas, node type vocabularies, and domain semantics to extension layers.

The format rests on 7 structural *boundaries*—design invariants that guarantee universal tooling—and every graph is validated against the CORE-16 contract: 9 structural rules, always enforced, plus 7 compositional rules, enforced when the graph declares a CORE vocabulary capability (Section 4.3).

4.1 Structural Foundation

The following field-schema fragment shows the 7 required node fields:

```
{
  "id": "string",
  "type": "string",
  "properties": {},
  "parent_id": "string | null",
  "contains": ["string"],
  "metric_level": "PREFIX_NAME",
  "dimension": "string"
}
```

The following field-schema fragment shows the 3 required edge fields:

```
{
  "from_id": "string",
  "to_id": "string",
  "relation": "string"
}
```

The 7 boundaries ensure universal tooling: any tool can read any TRUG.

1. **Core structure is invariant** — 7 node fields, 3 edge fields, always.
2. **Properties are open** — the `properties` object accepts any JSON content.
3. **Dimensions are declared** — hierarchical axes are explicit at the graph root.
4. **Hierarchy is explicit** — `parent_id` and `contains[]` are bidirectional.
5. **Edges are simple** — node-to-node with named relations, separate from hierarchy.
6. **Metric prefixes are standard** — 21 SI prefixes for hierarchical positioning.
7. **JSON structure** — valid JSON, UTF-8, language-agnostic.

4.2 Semantic Primitives

CORE defines 10 classes of semantic primitives. These are the atoms of meaning: every TRUG may use them and must interpret them identically.

Table 2: The 10 semantic primitive classes.

Class	Purpose	Graph Element
Operation	Data transformations	`properties.operation`
Relation	Named connections	Edge `relation`
Entity	Typed things	Node `type`
Constraint	Obligations/permissions	Constraint edges
Boundary	Hierarchy/reference	Hierarchy edges
Modifier	Constraints on entities	Node `properties`
Qualifier	Constraints on operations	Operation `properties`
Combinator	Graph structure control	Structural edges
Selector	Query scope	Node query scope
Reference	Back-references	Reference edges

The first 5 classes (Operation, Relation, Entity, Constraint, Boundary) define **what exists** in the graph. The latter 5 classes (Modifier, Qualifier, Combinator, Selector, Reference) define **how things compose**—they are the adjectives, adverbs, conjunctions, articles, and pronouns of the graph language.

4.3 Validation Rules

TRUGS enforces 16 validation rules—the CORE-16 contract. Rules 1–9 are structural and always enforced. Rules 10–16 are compositional and enforced when the graph declares a CORE vocabulary capability (`core_v1.0.0` or later).

Table 3: Validation rules. Rules 10–16 form the composition type system.

#	Rule	Type
1	Node ID uniqueness	Structural
2	Edge reference validity	Structural
3	Hierarchy consistency (bidirectional)	Structural
4	Metric level ordering (parent $\geq$ child)	Structural
5	Dimension declaration	Structural
6	Required fields present	Structural
7	Field type correctness	Structural
8	Extension declaration	Structural
9	Metric level format (`PREFIX_NAME`)	Structural
10	Subject-Operation compatibility	Compositional
11	Operation-Object compatibility	Compositional
12	Modifier-Entity compatibility	Compositional
13	Qualifier-Operation compatibility	Compositional
14	Constraint-Subject rule (modals require Actors)	Compositional
15	No double negation	Compositional
16	Reference scope (sentence-local)	Compositional

The compositional rules function as a **type system** for the graph. Rule 10 prevents artifacts from bearing obligations ("`DATA records SHALL SORT`" is invalid—data cannot have obligations). Rule 14 ensures that legal modals (`SHALL`, `MAY`, `SHALL_NOT`) attach only to Actor entities. These rules are not stylistic preferences; they are structural invariants that prevent semantically meaningless graphs.

5 TRUGS Language

The historical precedent for domain-specific constrained English is SQL, which achieved universal adoption not through syntactic novelty but by assigning formal semantics to familiar English words—`SELECT`, `FROM`, `WHERE`, `JOIN` [Chamberlin and Boyce, 1974]. Developers learned SQL not as a foreign language but as a constrained dialect of the language they already spoke.

The TRUGS Language follows this model. It is a constrained subset of English in which every valid sentence compiles deterministically to a TRUG graph, and every such graph decompiles to the original sentence. The majority of its vocabulary—FILTER, SORT, VALIDATE, PUBLIC, REQUIRED—consists of terms already in widespread use among software developers.

5.1 Vocabulary

The vocabulary consists of 233 executable words, organized into 9 parts of speech, plus an open sugar pattern:

Table 4: Vocabulary distribution by source domain (v2.0.0).

Part of Speech	Computation	Law	Shared	Total
Nouns	20	6	—	26
Verbs	67	13	—	80
Adjectives	29	10	—	39
Adverbs	12	7	—	19
Prepositions	14	4	—	18
Conjunctions	8	5	—	13
Articles	—	—	10	10
Pronouns	6	1	—	7
Level prefixes	—	—	21	21
Executable	**156**	**46**	**31**	**233**
Sugar	—	—	`'word` pattern	—

The 21 SI level prefixes (YOTTA ... BASE ... YOCTO) form the ninth part of speech: hierarchy markers used in the `metric_level` field and as bare-line zoom directives. Sugar is not a word list but a pattern (`'[a-z_]+`): any apostrophe-prefixed token compiles to nothing, so authors may freely annotate sentences for human readability without affecting the graph.

Each word belongs to exactly one part of speech. There are no dual-class words. Where a concept has both an action form and a relationship form, they are distinct words (e.g., ADMINISTER is the verb; GOVERNS is the preposition).

The vocabulary draws on the following sources:

- **Computation** (67%): words that programmers already know—FILTER, MAP, SORT, VALIDATE, ASYNC, PUBLIC, REQUIRED. Operations like `map`, `filter`, and `reduce` are recognized as universal higher-order functions across functional languages [Hutton, 2016]; the same design lineage runs through SQL's structured English [Chamberlin and Boyce, 1974] and the operation vocabularies of dataflow systems [Dennis and Misunas, 1975, Abadi et al., 2016]. These have deterministic meanings in every programming language.
- **Law** (20%): words that fill gaps where computation has no equivalent—SHALL (obligation), MAY (permission), NOTWITHSTANDING (override), REMEDY (consequence of breach), CONFIDENTIAL (access restriction). These terms come from a formal tradition: Hohfeld's fundamental legal conceptions [Hohfeld, 1919] established the precise relationships between rights, duties, privileges, and powers that underlie modern contract language. RFC 2119 [Bradner, 1997] later adopted the same distinctions for technical specifications.
- **Shared** (13%): domain-neutral scaffolding—the 10 articles (quantifiers and determiners) and the 21 SI level prefixes.[1]

5.2 Grammar

The grammar is defined in BNF [Backus, 1959]:

[1] Every count and percentage of the vocabulary in this paper is written by `tools/paper_vocab_counts.py` from `TRUGS_LANGUAGE/language.trug.json`, the language's own graph, at build time. The generator applies one rule beyond reading the file: a word node whose `source` field is empty—at this version, the articles—is counted as shared, since those words belong to neither computation nor law.

```
program        := preamble* (sentence | level_directive)+
preamble       := WHEREAS clause "."
sentence       := clause (CONJUNCTION clause)* "."
                | definition "."
                | leading_keyword_directive "."
clause         := subject verb_phrase (object_phrase)*
definition     := DEFINE value AS noun_phrase
leading_keyword_directive
               := ASSERTION_KEYWORD clause (CONJUNCTION clause)*
ASSERTION_KEYWORD
               := ASSERT | ASSERT_NOT | INVARIANT
subject        := noun_phrase | PRONOUN
noun_phrase    := [ARTICLE] [ADJECTIVE]* NOUN [identifier]
noun_list      := noun_phrase (AND noun_phrase)*
verb_phrase    := [modal] VERB adverb_phrase*
adverb_phrase  := ADVERB [value]
object_phrase  := prep_phrase (AND prep_phrase)*
                | noun_phrase | PRONOUN
prep_phrase    := PREPOSITION noun_list
modal          := SHALL | SHALL_NOT | MAY
```

The grammar enforces that every clause has a subject and a verb, that conjunctions connect complete clauses, and that pronouns resolve within the current sentence. A `level_directive` is a bare SI prefix on its own line—a zoom marker for the reader that compiles to nothing. A leading-keyword directive places an assertion keyword before the subject (`INVARIANT RESOURCE tracker SHALL REMAIN open.`); this production is part of the v2 grammar specification, while the reference parser currently implements the subject-first productions, with directive support scheduled.[2]

5.3 Composition Rules

Not every grammatically valid sentence is semantically valid. The composition rules define which parts of speech can combine with which:

- **Subject-Verb**: Only Actors can be subjects of most verbs. Artifacts can only be subjects of comparison verbs (`EXISTS`, `EQUALS`, `EXCEEDS`).
- **Verb-Object**: Transforms act on Artifacts. Permit/Prohibit verbs target Actors. Resolve verbs target Outcomes.
- **Modal-Subject**: `SHALL`, `MAY`, `SHALL_NOT` require Actor subjects. Data cannot bear obligations.
- **Adjective-Noun**: Type adjectives (`STRING`, `INTEGER`) only modify Artifacts. State adjectives are universal.
- **Adjective ordering**: Fixed stacking order: Quantity > Priority > State > Access > Type > Noun. This follows the cross-linguistic pattern identified by Sproat and Shih [1991], where absolute properties (type, color) are closer to the noun than relative properties (quantity, size).
- **Adverb-Verb**: Timing/Repetition modify any verb except Bind. Degree modifies authority verbs.

These rules map directly to validation rules 10–16 in the graph specification, establishing a formal correspondence between grammatical validity and graph validity.

6 Worked Examples

To illustrate the compilation process concretely, we present three examples of increasing complexity. Each follows the same pipeline: a sentence is written, parsed token-by-token against the grammar and composition rules, compiled to a JSON graph, and decompiled back to the original sentence.

[2]Implementation status as of this revision: the published `trug` CLI parses the subject-first grammar; the leading-keyword directive production is specified and pending in the reference parser.

6.1 Example 1: Simple Obligation

Sentence:

```
PARTY system SHALL VALIDATE ALL PENDING RECORD.
```

Parse:

Token	Part of Speech	Subcategory	Rule Check
PARTY	Noun	Actor	✓ subject
system	identifier	—	✓
SHALL	Modal	Obligate	✓ Actor subject (Rule 14)
VALIDATE	Verb	Obligate	✓
ALL	Article	Universal	✓
PENDING	Adjective	State	✓ State on Artifact (Rule 12)
RECORD	Noun	Artifact	✓ object of Obligate (Rule 11)

Compiled graph: the SHALL edge links the actor to the validation operation; an ACTS_ON edge carries the object phrase, including ALL PENDING. The full compiler output is in Appendix E.2.

Decompiled: PARTY system SHALL VALIDATE ALL PENDING RECORD. — identical to input. ✓

6.2 Example 2: Multi-Party with Retry and Error Handling

Sentence:

```
PARTY client SHALL REQUEST PARTY server.
PARTY server SHALL RESPOND PROMPTLY WITHIN 30s
  OR PARTY client MAY RETRY BOUNDED 3
  THEN HANDLE THE ERROR.
```

Parse (sentence 2):

Token	Part of Speech	Subcategory	Rule Check
PARTY	Noun	Actor	✓ subject
server	identifier	—	✓
SHALL	Modal	Obligate	✓ Actor (Rule 14)
RESPOND	Verb	Move	✓
PROMPTLY	Adverb	Timing	✓ Timing on Move (Rule 13)
WITHIN	Adverb	Timing	✓
30s	value	duration	✓ WITHIN requires duration
OR — alternative conjunction			
PARTY	Noun	Actor	✓ new subject
client	identifier	—	✓
MAY	Modal	Permit	✓ Actor (Rule 14)
RETRY	Verb	Control	✓
BOUNDED	Adverb	Repetition	✓ Repetition on Control (Rule 13)
3	value	integer	✓ BOUNDED requires integer
THEN — sequential conjunction			
HANDLE	Verb	Resolve	✓ inherits subject (client)
THE	Article	Specific	✓
ERROR	Noun	Outcome	✓ object of Resolve (Rule 11)

This example demonstrates: two actors communicating, adverbs with values (PROMPTLY, WITHIN 30s, BOUNDED 3), conjunctions (OR for fallback, THEN for sequence), modal permission (MAY) versus obligation (SHALL), and subject inheritance across a THEN chain.

The compiled graph contains 4 nodes (2 actors, 2 operations) and 5 edges (SHALL, MAY, OR, THEN, FEEDS). Each adverb-value pair becomes an operation property. Each conjunction becomes a structural edge.

6.3 Example 3: Prohibition with Exception and Authentication

Sentence:

```
NO PARTY SHALL WRITE CONFIDENTIAL RESOURCE
  EXCEPT PARTY owner MAY WRITE CONFIDENTIAL RESOURCE
    PROVIDED_THAT PARTY owner AUTHENTICATE TO SERVICE auth.
```

Parse:

Token	Part of Speech	Subcategory	Rule Check
NO	Article	Negative	✓ zero instances
PARTY	Noun	Actor	✓
SHALL	Modal	Obligate	✓ Actor (Rule 14)
WRITE	Verb	Move	✓
CONFIDENTIAL	Adjective	Access	✓ Access on Artifact (Rule 12)
RESOURCE	Noun	Artifact	✓ object of Move (Rule 11)
EXCEPT — carve-out (from law)			
PARTY	Noun	Actor	✓
owner	identifier	—	✓
MAY	Modal	Permit	✓
WRITE	Verb	Move	✓
CONFIDENTIAL	Adjective	Access	✓
RESOURCE	Noun	Artifact	✓
PROVIDED_THAT — prerequisite (from law)			
PARTY	Noun	Actor	✓
owner	identifier	—	✓
AUTHENTICATE	Verb	Move	✓
TO	Preposition	Flow	✓
SERVICE	Noun	Actor	✓
auth	identifier	—	✓

This example demonstrates the power of law-derived vocabulary. The sentence reads like a legal clause:

- `NO PARTY SHALL` — universal prohibition (computation has no equivalent)
- `EXCEPT` — carve-out from the prohibition (from contract law)
- `MAY` — permission granted to a specific party
- `PROVIDED_THAT` — prerequisite condition (from contract law)

This is three levels of nesting: prohibition → exception → prerequisite. Each level is a complete clause connected by a law-derived conjunction. The compiled graph represents this as conditional edges: the EXCEPT edge overrides the prohibition, but only when the PROVIDED_THAT edge's condition (authentication) is satisfied.

A natural language version of this same requirement—"No one can write confidential resources except the owner, but only if they've authenticated first"—is ambiguous. Does "first" mean before writing or before any access? Does "no one" include system processes? In TRUGS, `NO PARTY` means zero Actor entities. `CONFIDENTIAL RESOURCE` means Artifact with Access modifier. `AUTHENTICATE TO SERVICE auth` means Move verb to a specific Actor. No ambiguity.

6.4 Observations

These three examples progress from a single clause to three nested clauses, demonstrating several properties of the system. First, each token maps deterministically to a single grammar element; the parse is mechanical, requiring no inference or disambiguation. Second, the composition rules (Rules 10–16) reject semantically invalid combinations at parse time—before any execution occurs. Third, the legal vocabulary (`SHALL`, `EXCEPT`, `PROVIDED_THAT`) provides expressive power for governance

patterns that computational vocabulary alone cannot capture. Finally, decompilation of the compiled graph reproduces the original sentence in each case, providing empirical evidence of the round-trip property.

The complete token-by-token analysis for all 30 examples is available in the specification repository.

7 Bidirectional Compilation

The central claim of TRUGS is that the sentence and the graph are lossless renderings of the same artifact. We define the compilation and decompilation functions and characterize their round-trip behavior.

7.1 Sentence to Graph

Every sentence element compiles to exactly one graph element:

Table 5: Compilation mapping from sentence elements to graph elements.

Sentence Element	Graph Element
`NOUN` + identifier	Node: `{id: identifier, type: NOUN}`
`ADJECTIVE` on noun	Node property
`ARTICLE` on noun	Query scope
`VERB`	Operation node
Modal on verb	Constraint edge
`ADVERB` on verb	Operation property
`PREPOSITION`	Edge with typed relation
`CONJUNCTION`	Graph structure
`PRONOUN`	Back-reference edge

7.2 Graph to Sentence

Decompilation walks the graph in topological order and emits words by reversing the compilation table. Every graph element maps to exactly one word. The word order is determined by the graph structure: subjects precede verbs, verbs precede objects, conjunctions separate clauses.

7.3 Round-Trip Property

Let C denote the compilation function (sentence $\rightarrow$ graph) and D denote the decompilation function (graph $\rightarrow$ sentence). We claim that for any valid TRUGS sentence s:

$$D(C(s)) = s$$

and for any TRUGS graph g produced by compilation:

$$C(D(g)) = g$$

This property follows from the design of the mapping: each sentence element corresponds to exactly one graph element, and the BNF grammar is constructed to be unambiguous (no valid sentence admits two distinct parse trees). A sentence that fails to round-trip is, by definition, not a valid TRUGS sentence.

We have verified this property empirically with the released tool on the 30 examples of the specification's example file, of which 29 decompile to their source text (Section 9.1). A mechanized formal proof—for example, in Coq or Lean—would provide a stronger guarantee and remains an important direction for future work.

8 The Opening TRUG

Every session between a human and a computational agent begins by loading an **opening TRUG**: a TRUG whose type is `LANGUAGE` that defines the valid vocabulary, grammar rules, and constraints for that session.

8.1 Structure

An opening TRUG contains four types of nodes:

- **WORD** nodes: every valid word with its part of speech, subcategory, and exact definition.
- **GRAMMAR_RULE** nodes: composition rules defining which parts of speech combine.
- **CONSTRAINT** nodes: structural limits (e.g., maximum nesting depth).
- **DOMAIN** nodes: domain-specific vocabulary extensions.

8.2 Self-Description

The TRUGS Language definition is itself a TRUG (`language.trug.json`). This recursive property—the language that describes TRUGs is a TRUG—ensures that:

- The language definition validates against the same rules it defines.
- Version control is built in: each language version is a specific graph (the 233-word vocabulary is `2.0.0`).
- Domain extensibility is natural: a medical opening TRUG adds `DIAGNOSE`, `PRESCRIBE` as WORD nodes.
- A computational agent can validate its own understanding by checking its behavior against the opening TRUG.

8.3 Communication Model

The opening TRUG establishes a shared contract between all participants in a conversation:

Table 6: Three communication modes, one language.

Mode	Direction	Mechanism
Human $\rightarrow$ LLM	Human writes sentence	LLM compiles to graph, executes
LLM $\rightarrow$ Human	LLM produces graph	Decompiles to sentence human reads
LLM $\rightarrow$ LLM	LLM A emits sentence	LLM B compiles and executes

The same 233 words mean the same thing to every participant because the opening TRUG defines them. The model does not rely on training to understand `FILTER`—it reads the definition from the graph: "select items satisfying a predicate; formally, given collection C and predicate P, $\texttt{FILTER}(C, P) = \{x \in C \mid P(x) = \text{true}\}$."

8.4 Language Agnosticism

Although the reference vocabulary is English, TRUGS is language-agnostic. The grammar is universal—it defines structural relationships between parts of speech, not between English words. The vocabulary is loaded from the opening TRUG. A Chinese opening TRUG replaces English words with Chinese equivalents:

This is not translation. These are independently evolved terms with formally established meanings in their respective legal systems and programming cultures. Chinese contract law defines 应当 (yīngdāng) as mandatory obligation—the same concept English common law assigns to *shall* and RFC 2119 assigns to `MUST`. Every major legal tradition independently developed words for obligation, prohibition, and permission because these are universal governance concepts. Similarly, every

Table 7: Cross-language vocabulary: the same concepts exist independently in law and computation across languages.

Concept	English	Chinese	Spanish	Class	Source
Obligation	SHALL	应当 (yīngdāng)	DEBERÁ	Modal	Law
Prohibition	SHALL_NOT	不得 (bùdé)	NO DEBERÁ	Modal	Law
Permission	MAY	可以 (kěyǐ)	PODRÁ	Modal	Law
Exception	EXCEPT	除非 (chúfēi)	SALVO	Conjunction	Law
Filter	FILTER	筛选 (shāixǎn)	FILTRAR	Verb	Computation
Validate	VALIDATE	验证 (yànzhèng)	VALIDAR	Verb	Computation
Record	RECORD	记录 (jìlù)	REGISTRO	Noun	Computation

programming language has a concept of filter, sort, validate, and return—because these are universal computational operations.

The grammar (BNF) is invariant across languages. The composition rules (which parts of speech combine with which) are invariant. The compilation mapping (sentence element → graph element) is invariant. Only the surface vocabulary changes. This means:

- A Chinese-speaking developer loads a Chinese opening TRUG and writes TRUGS in Chinese.
- The compiled graph is identical to the English version—same nodes, same edges, same structure.
- An English-speaking LLM can read the graph and decompile it to English.
- Cross-language communication is automatic: both parties share the graph, each renders it in their language.

Additionally, TRUGS defines an open **sugar** pattern—apostrophe-prefixed tokens (`'word`) that compile to nothing. Sugar makes sentences read like natural prose without affecting the graph: `'of`, `'is`, `'that`, `'please`, or domain annotations such as `'postgres`. A sentence with sugar and the same sentence without it produce the identical graph. A localized vocabulary would pair the same mechanism with its own particle set—for Chinese, 的 (de), 是 (shì), 了 (le)—particles that make sentences flow naturally but carry no semantic weight in the graph.

9 Evaluation

9.1 Grammar Validation: the Parsed Examples

The specification's example file (`TRUGS_LANGUAGE/SPEC_examples.md`) holds 30 numbered examples, from simple obligations and nested conditions through event-driven patterns, concurrent actors, batch processing, rate limiting, self-description and a complete ETL pipeline. The build runs the released tool on every one of them (`tools/check_paper_trl.py`): each is compiled, and each compiled graph is decompiled and compared with the source text.

Table 8: Grammar validation with the released tool over the numbered examples.

Metric	Count
Examples in the file	30
Compiled by the released tool	29
Decompiled to the source text	29
Not accepted (example numbers)	30

The examples were written before the tool existed, and the file records what writing them taught: the words the vocabulary lacked, the grammar productions added, the one reclassification and the rule revisions are listed in its closing table. Each early failure pointed to a specific vocabulary gap or grammar limitation, and the process of writing examples, parsing them, diagnosing failures

and patching the specification was the primary mechanism by which the language and the graph specification co-evolved. The example the tool does not accept uses the metric-level value form, which the file marks as reserved.

9.2 Validator: the CORE-16 Contract Under Test

At specification time, the reference validator implemented all 16 CORE rules in a single Python file (520 lines, zero external dependencies), with 47 unit tests covering:

- All 9 structural rules (pass and fail cases)
- All 7 compositional rules (pass, fail, and edge cases)
- Opt-in behavior (compositional rules fire only when a CORE vocabulary capability is declared)
- File I/O (valid JSON, invalid JSON, missing file)

The validator was run against all 21 TRUG files then in the specification repository; all 21 passed with 0 errors. Since the 2.0 release, the reference implementation ships as the installable `trugs-tools` package (the `trug` CLI) and the specification repository is spec-only; the CORE-16 contract itself is unchanged.

9.3 Example Corpus Validation

During development, the specification was exercised against 20 example TRUGs across 7 structurally distinct domains, each representing a different class of structured information (the current repository curates one exemplar per domain):

Table 9: Example corpus: 7 domains demonstrating TRUGS generality.

Domain	What it models	Files	Key structure
Knowledge	Ontologies, taxonomies	3	CLASS → ENTITY → INSTANCE
Living	Real-time agent queries	3	QUERY → TOOL → ANSWER
Memory	LLM persistent memory	1	MODULE → DATA + REFERENCES edges
Nested	Subgraph composition	3	TASK → SUBGRAPH → RESULT
Research	Claims and evidence	2	CONCEPT → CLAIM → SOURCE
Web	Site structure	4	SITE → PAGE → SECTION
Writer	Document structure	4	DOCUMENT → SECTION → PARAGRAPH

These domains span ontology modeling, agent orchestration, persistent memory, hierarchical composition, evidence-based research, web architecture, and document authoring. The same 7 boundaries, 16 validation rules, and composition type system govern all of them. No domain-specific validation code was written — CORE is sufficient.

During development, the validator discovered 231 errors across 19 of these example files — all systematic (missing `contains:[]` on leaf nodes, inconsistent bidirectional hierarchy). These were not contrived test failures; they were structural errors in hand-authored examples that manual review missed. After automated repair, all 21 files passed validation with 0 errors.

This exercise demonstrates two things: first, that the validator catches real errors in real TRUGs across diverse domains. Second, that structural validation is necessary — even specification authors make systematic errors that only mechanical checking finds.

9.4 Working System: LLM Memory

As a practical demonstration, we built a TRUG-based memory system for LLMs with 5 operations:

- `init` — create an empty memory TRUG (declares the CORE vocabulary capability)
- `remember` — add a memory node with type, tags, and source
- `recall` — query by keyword, type, recency, or retrieve all

- `forget` — remove a memory and its associated edges
- `associate` — create typed edges between memories

The memory graph validates against all 16 CORE rules. Memories are `DATA` nodes under a `MODULE` root. Associations are `REFERENCES` edges. The system requires zero external dependencies and produces valid TRUGs by construction.

This corresponds to Section 6.2 ("Dynamic Knowledge Memory for Agentic Systems") of the recent KG construction survey [Bian, 2025], which identifies persistent structured memory as a key future direction. We implement it as a direct consequence of the specification.

10 Applications

While the preceding sections focus on the specification and its validation, we briefly illustrate four application domains where TRUGS provides immediate practical utility. Each example shows a TRUGS Language sentence expressing a real-world pattern.

10.1 LLM Persistent Memory

LLMs lack persistent memory across sessions. TRUGS provides a natural structure for it: memories are `DATA` nodes, associations are `REFERENCES` edges, and the memory graph validates against CORE. The reference implementation (Section 9.4) demonstrates this directly.

```
WHEREAS DATA decision REFERENCES DATA context.
PARTY assistant SHALL APPEND DATA decision TO RECORD memory.
PARTY assistant MAY READ ALL ACTIVE DATA FROM RECORD memory
  SUBJECT_TO DATA query.
```

The graph persists between sessions. Reading the memory back is a graph query, not a text search—it follows typed edges to return structurally related memories, not keyword matches.

10.2 Inter-Agent Communication

When multiple LLMs collaborate, they currently exchange natural language, inheriting all its ambiguity. With TRUGS, agents load the same opening TRUG and communicate in compiled sentences. Each agent can verify that it parsed the same graph the sender intended—because the sentence *is* the graph.

```
PARTY planner SHALL SPLIT THE MESSAGE TO AGENT researcher AND AGENT
    writer.
EACH AGENT SHALL HANDLE INPUT PARALLEL
  THEN MERGE RESULT TO PARTY planner.
PARTY planner SHALL RESPOND TO PARTY user.
```

The fan-out/fan-in pattern is encoded in the topology. No agent can misinterpret "handle in parallel"—`PARALLEL` is an adverb with a single mechanical definition, not a suggestion.

10.3 Contract and Compliance Modeling

Legal documents are governance structures: obligations, permissions, prohibitions, exceptions, and remedies. TRUGS encodes these directly because its vocabulary inherits from legal language. A compliance rule that would occupy a paragraph of prose compiles to a graph where every obligation is traceable to an actor and every exception is a typed edge.

```
NO PARTY SHALL WRITE CONFIDENTIAL RESOURCE
  EXCEPT PARTY data-controller
    PROVIDED_THAT PARTY data-controller AUTHENTICATE TO SERVICE
        compliance-gateway.
IF ANY PARTY WRITE CONFIDENTIAL RESOURCE
  THEN THE REMEDY DEPENDS_ON PARTY data-controller.
```

This is a data protection rule: no one may write confidential data except the data controller, and only after authenticating with the compliance gateway. The compiled graph makes every element auditable—which actor, which obligation, which exception, which remedy. A compliance auditor can traverse the graph without reading prose.

10.4 Codebase Understanding

A software repository can be modeled as a TRUG: modules, functions, dependencies, and ownership as typed nodes and edges. An LLM reads the graph to understand the architecture without parsing every source file.

```
MODULE api CONTAINS FUNCTION authenticate.
MODULE api CONTAINS FUNCTION authorize.
MODULE api CONTAINS FUNCTION handle-request.
FUNCTION handle-request REQUIRE FUNCTION authenticate.
FUNCTION authorize REQUIRE FUNCTION authenticate.
PARTY security-team SHALL ADMINISTER MODULE api.
```

The graph encodes what exists (modules, functions), how they depend on each other (`REQUIRE`), who is responsible (`ADMINISTER`), and what the structure contains (`CONTAINS`). An LLM navigating a new codebase reads this graph in milliseconds—versus minutes scanning documentation that may be out of date. This pattern is implemented by the published `trugs-folder` package: `trug-a-folder` renders a repository's filesystem into a `folder.trug.json` map, checks it against the tree, and keeps the two synchronized.

11 Discussion

11.1 Why Not Natural Language Alone?

A natural objection is that LLMs already process English effectively. However, processing and *verification* are distinct capabilities. An LLM can respond to "handle the data appropriately," but no mechanism exists for either party to verify semantic agreement on "appropriately." Prompt engineering reduces this gap probabilistically through context and examples, but probabilistic alignment is not the same as structural alignment.

In TRUGS, ambiguity is a compilation error. A sentence either parses against the grammar and satisfies the composition rules, or it is rejected before execution. This provides a *verifiable* communication channel: if the sentence compiles, its meaning is the graph—and both parties can inspect the graph.

11.2 Why Not Structured Formats Directly?

While JSON graphs are precise, they are not practical for direct human authoring. A TRUGS sentence such as `PARTY system SHALL FILTER ALL ACTIVE RECORD` communicates the same information as a JSON node-and-edge structure, but is readable at a glance. The language serves as a human interface to the underlying graph representation; both are necessary, and neither is sufficient alone.

11.3 Lexical Unambiguity

An early design decision allowed words to function as both verbs and prepositions depending on syntactic position (e.g., `GOVERNS` as both "establish authority" and as an authority edge relation). This was eliminated in favor of distinct lexemes: `ADMINISTER` (verb) and `GOVERNS` (preposition). The governing principle is that every word belongs to exactly one part of speech. Context-dependent lexical classification would reintroduce the ambiguity the language is designed to eliminate.

11.4 Adoption Considerations

Historically, domain-specific languages succeed when they draw on vocabulary their target users already know. SQL achieved widespread adoption because database developers recognized `SELECT`,

`WHERE`, and `JOIN` as English words with familiar meanings [Chamberlin, 2012]. TRUGS follows this pattern: its vocabulary overlaps substantially with terminology already used by software developers (`FILTER`, `VALIDATE`, `ASYNC`) and legal practitioners (*shall*, *notwithstanding*, *provided that*). The open sugar pattern further reduces the perceived distance from natural English by allowing grammatical particles that the compiler strips without effect.

11.5 Limitations

- **Vocabulary coverage**: 233 words cannot express everything. The vocabulary is closed by design—when a domain concept seems to call for a new word, the first move is to express it with existing vocabulary; domain-specific concepts (e.g., medical diagnoses, financial instruments) that genuinely exceed it require opening TRUG extensions. The base vocabulary covers computation and governance; other domains are future work.
- **No runtime semantics**: TRUGS defines what a valid program is, not how it executes. The graph is an AST, not an interpreter. Execution semantics are intentionally outside scope—different runtime environments may execute the same graph differently.
- **Empirical, not formal, verification**: The round-trip property is demonstrated by the 29 examples the released tool round-trips, not by a formal proof. A mechanized proof (e.g., in Coq or Lean) would provide a stronger foundation.
- **No user study**: We have not yet measured human comprehension, writing speed, or error rates when using TRUGS versus natural language or JSON. This is planned future work.

12 Conclusion

We have presented TRUGS, a formalized language in which constrained human-language sentences and directed graphs are bidirectional, empirically lossless renderings of the same artifact. By drawing vocabulary from computation and law—two domains with established formal traditions—TRUGS avoids the need to invent new semantics: the precision is inherited from centuries of professional practice. The language-agnostic grammar ensures that the same compilation and validation rules apply regardless of the surface language.

Several directions remain open. A mechanized proof of the round-trip property would strengthen the formal foundation. User studies measuring comprehension, writing speed, and error rates against natural language and JSON would quantify the practical benefit. Extending the vocabulary to additional domains (medicine, finance, manufacturing) through the opening TRUG mechanism would test the extensibility claim at scale.

TRUGS does not aim to replace natural language for human communication or structured formats for machine storage. Rather, it addresses the gap between them: providing a representation precise enough for machine validation, readable enough for human authoring, and formal enough to compile. The specification and all examples are released under the Apache 2.0 license, and the reference toolchain is published as installable packages (`trugs-tools`, `trugs-folder`), to facilitate independent evaluation and adoption.

A Note on the Name

The word *trug* is Welsh in origin, denoting a shallow woven basket used for carrying garden produce. The name was chosen before the acronym (Traceable Recursive Universal Graph) was discovered to fit—a coincidence that proved apt.

A woven basket is structure in service of content. The weave determines the basket's strength, shape, and capacity, but no one values a basket for its weave. They value it for what it carries. TRUGS follows this principle: the graph structure—nodes, edges, hierarchy, validation rules—is the weave. What matters is the content it carries: the knowledge, the operations, the obligations, the meaning. The specification is meticulous about structure precisely so that users can focus on substance.

The recursive property of TRUGS extends this metaphor. The language definition is a TRUG. The memory system stores TRUGs. The validator validates TRUGs against rules defined in a TRUG. The opening TRUG that defines the vocabulary is loaded from a TRUG. Each layer is a basket carrying

another basket—the weave that holds the weave that carries the content. In the idiom of the project: TRUGs all the way down.

AI Assistance Disclosure

This paper was developed with the assistance of Anthropic Claude (Claude Opus 4.6; the citation-audit revision with Claude Fable 5). Claude served as a development partner throughout the project: co-designing the language specification, writing and debugging the validator, stress-testing the grammar against 30 examples, and drafting sections of this paper. All scientific claims, design decisions, and architectural choices are the author's. The author takes full responsibility for the content. Claude is not listed as a co-author as it cannot take responsibility for published work.

References

Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 265–283, 2016.

Anthropic. Model context protocol specification. `https://modelcontextprotocol.io/specification/2025-11-25`, 2025.

John W. Backus. The syntax and semantics of the proposed international algebraic language of the Zurich ACM-GAMM conference. *Proceedings of the International Conference on Information Processing*, pages 125–131, 1959.

Tim Berners-Lee, James Hendler, and Ora Lassila. The semantic web. *Scientific American*, 284(5): 34–43, 2001.

Haonan Bian. LLM-empowered knowledge graph construction: A survey. In *Proceedings of ICAIS 2025*, 2025. arXiv:2510.20345.

Scott Bradner. Key words for use in RFCs to indicate requirement levels. RFC 2119, 1997. URL `https://datatracker.ietf.org/doc/html/rfc2119`.

Donald D. Chamberlin. Early history of SQL. *IEEE Annals of the History of Computing*, 34(4): 78–82, 2012.

Donald D. Chamberlin and Raymond F. Boyce. SEQUEL: A structured English query language. In *Proceedings of the 1974 ACM SIGFIDET Workshop on Data Description, Access and Control*, pages 249–264. ACM, 1974.

Peter Clark, Phil Harrison, Tom Jenkins, John Thompson, and Richard Wojcik. Acquiring and using world knowledge using a restricted subset of english. In *Proceedings of FLAIRS*, 2005.

Anne Cregan, Rolf Schwitter, and Thomas Meyer. Sydney OWL syntax — towards a controlled natural language syntax for OWL 1.1. In *Proceedings of OWLED 2007*, 2007.

Jack B. Dennis and David P. Misunas. A preliminary architecture for a basic data-flow processor. In *Proceedings of the 2nd Annual Symposium on Computer Architecture (ISCA)*, pages 126–132, 1975.

D. Dori, I. Reinhartz-Berger, and A. Sturm. OPCAT—a bimodal CASE tool for object-process based system development. In *Proceedings of the 5th International Conference on Enterprise Information Systems (ICEIS 2003)*, pages 286–291, 2003.

Norbert E. Fuchs, Kaarel Kaljurand, and Tobias Kuhn. Attempto controlled english for knowledge representation. In *Reasoning Web*, pages 104–124. Springer, 2008.

Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.

David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, 1987.

Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973.

Wesley Newcomb Hohfeld. *Fundamental Legal Conceptions as Applied in Judicial Reasoning*. Yale University Press, 1919.

Graham Hutton. *Programming in Haskell*. Cambridge University Press, 2nd edition, 2016. Defines map, filter, foldr as universal higher-order functions across functional languages.

Richard M. Karp and Raymond E. Miller. Properties of a model for parallel computations: Determinacy, termination, queueing. *SIAM Journal on Applied Mathematics*, 14(6):1390–1411, 1966.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

Tobias Kuhn. A survey and classification of controlled natural languages. *Computational Linguistics*, 40(1):121–170, 2014. doi: 10.1162/COLI_a_00168.

LangChain AI. LangGraph: Build stateful, multi-actor applications with LLMs. `https://github.com/langchain-ai/langgraph`, 2024.

David Martin, Mark Burstein, Jerry Hobbs, Ora Lassila, Drew McDermott, Sheila McIlraith, Srini Narayanan, Massimo Paolucci, Bijan Parsia, Terry Payne, Evren Sirin, Naveen Srinivasan, and Katia Sycara. OWL-S: Semantic markup for web services. W3C Member Submission, 22 November 2004, 2004. URL `https://www.w3.org/submissions/2004/SUBM-OWL-S-20041122/`.

Richard Montague. The proper treatment of quantification in ordinary english. In Julius Hintikka, Julius Moravcsik, and Patrick Suppes, editors, *Approaches to Natural Language*, pages 221–242. D. Reidel, 1973.

João Moura. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. `https://github.com/crewAIInc/crewAI`, 2023.

Neo4j, Inc. The Neo4j graph database. `https://neo4j.com`, 2024.

Imran Ahsan Nyazee. Comparison between Islamic law, civil law and common law. *SSRN Electronic Journal*, 2022. doi: 10.2139/ssrn.4163738.

Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.

Carl Adam Petri. *Kommunikation mit Automaten*. PhD thesis, Technische Universität Darmstadt, Bonn, 1962. English translation: Communication with Automata, 1966.

M. Ross Quillian. Semantic memory. In Marvin Minsky, editor, *Semantic Information Processing*, pages 216–270. MIT Press, Cambridge, MA, 1968.

Rolf Schwitter. English as a formal specification language. In *Proceedings of DEXA Workshops*, pages 228–232. IEEE, 2002.

Rolf Schwitter. Specifying and verbalising answer set programs in controlled natural language. arXiv:1804.10765, 2018. URL `https://arxiv.org/abs/1804.10765`. Presented at the 34th International Conference on Logic Programming (ICLP 2018), Oxford.

Rolf Schwitter and Norbert E. Fuchs. Attempto — from specifications in controlled natural language towards executable specifications. arXiv:cmp-lg/9603004, 1996. URL `https://arxiv.org/abs/cmp-lg/9603004`. Presented at the EMISA Workshop, Tutzing, May 1996.

Richard Sproat and Chilin Shih. The cross-linguistic distribution of adjective ordering restrictions. In Carol Georgopoulos and Roberta Ishihara, editors, *Interdisciplinary Approaches to Language*, pages 565–593. Kluwer Academic Publishers, 1991.

Adrian Tam et al. SLOT: Structuring the output of large language models. In *Proceedings of EMNLP 2025 Industry Track*, 2025.

Georg Henrik von Wright. Deontic logic. volume 60, pages 1–15. 1951.

W3C. OWL 2 web ontology language document overview. W3c recommendation, World Wide Web Consortium, 2012.

W3C. RDF 1.1 concepts and abstract syntax. W3c recommendation, World Wide Web Consortium, 2014.

W3C. JSON-LD 1.1. W3c recommendation, World Wide Web Consortium, 2020.

Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. In *Proceedings of ICML 2024 Workshop on Structured Probabilistic Inference*, 2023. arXiv:2307.09702.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Yevgeny Yaroker, Valeriya Perelman, and Dov Dori. An OPM conceptual model-based executable simulation environment: Implementation and evaluation. *Systems Engineering*, 16(4):381–390, 2013. doi: 10.1002/sys.21235.

Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 15–28, 2012.

Zhuoxun Zheng, Baifan Zhou, Dongzhuoran Zhou, Xianda Zheng, Gong Cheng, Ahmet Soylu, and Evgeny Kharlamov. Executable knowledge graphs for machine learning: A Bosch case of welding monitoring. In U. Sattler et al., editors, *The Semantic Web – ISWC 2022*, volume 13489 of *Lecture Notes in Computer Science*, pages 791–809. Springer, 2022. doi: 10.1007/978-3-031-19433-7_45.

A Complete ETL Pipeline Example

The following program demonstrates TRUGS Language expressing a realistic ETL (Extract, Transform, Load) pipeline with 4 actors, error handling, authentication, and authority:

```
DEFINE "raw-event" AS STRING DATA.
DEFINE "clean-event" AS VALID OBJECT DATA.
DEFINE "event-store" AS IMMUTABLE ENDPOINT.

WHEREAS SERVICE kafka FEEDS STREAM raw-events.
WHEREAS ENDPOINT event-store REQUIRE MODULE postgres.

PARTY ingester SHALL READ EACH DATA raw-event FROM STREAM raw-events
    WITHIN 100ms
  OR PARTY ingester SHALL RETRY BOUNDED 5 WITHIN 30s
  OR THROW EXCEPTION.

PARTY transformer SHALL VALIDATE THE DATA raw-event
    SUBJECT_TO INTERFACE event-schema
  THEN MAP RESULT TO DATA clean-event
```

```
  THEN BATCH RESULT 500.

PARTY loader SHALL WRITE EACH RESULT TO ENDPOINT event-store
  OR RETRY BOUNDED 3 WITHIN 60s.

IF PARTY loader THROW EXCEPTION
  THEN PARTY loader SHALL SEND ERROR TO PARTY monitor
  THEN PARTY monitor SHALL SEND MESSAGE TO PARTY oncall.

NO PARTY SHALL WRITE ENDPOINT event-store
  EXCEPT PARTY loader
    PROVIDED_THAT PARTY loader AUTHENTICATE TO SERVICE auth.

PARTY monitor SHALL ADMINISTER PARTY ingester AND PARTY transformer
    AND PARTY loader.
```

This program is 11 sentences, uses 3 definitions, 2 preambles, 4 actors, retry with timeout, error handling with escalation, authentication, and authority hierarchy. Every word is from the 233-word vocabulary. Every sentence parses against the BNF grammar. The compiled graph validates against all 16 CORE rules.

B Complete Vocabulary Glossary

This appendix is the word list of the canonical specification (`TRUGS_LANGUAGE/SPEC_vocabulary.md`, v2.0.0), from which `language.trug.json` is itself generated. The word rows were generated from that specification; the counts in the headings and in the summary table are written at build time from `language.trug.json` by `tools/paper_vocab_counts.py`. Every word is listed exactly once, alphabetized within its part of speech. The count here is the count: 233 executable words. Sugar is not counted—it is an open pattern (`'[a-z_]+`), not a word list.

B.1 Nouns (26)

Word	Sub	Src	Definition
AGENT	Actor	Law	An entity authorized to act on behalf of another
DATA	Artifact	Comp	A typed value: input, output, or intermediate state
ENDPOINT	Boundary	Comp	A named, addressable point of interaction
ENTRY	Boundary	Comp	The point where data enters a scope
ERROR	Outcome	Comp	A named failure condition with a cause
EXCEPTION	Outcome	Comp	An error that interrupts normal flow
EXIT	Boundary	Comp	The point where data leaves a scope
FILE	Artifact	Comp	A named, persistent byte sequence
FUNCTION	Actor	Comp	A callable unit that accepts input and returns output
INSTRUMENT	Artifact	Law	A formal document that creates or records rights
INTERFACE	Boundary	Comp	A contract defining what operations a thing exposes
JURISDICTION	Boundary	Law	The scope within which authority is valid
MESSAGE	Artifact	Comp	A discrete unit of communication between actors
MODULE	Container	Comp	A self-contained unit of related functionality
NAMESPACE	Container	Comp	A named scope that prevents identifier collision
PARTY	Actor	Law	A bound entity capable of obligations and permissions
PIPELINE	Container	Comp	A container of ordered operations data flows through
PRINCIPAL	Actor	Law	The entity on whose behalf an agent acts
PROCESS	Actor	Comp	An executing unit of work with its own state
RECORD	Artifact	Comp	A single structured data item with named fields
REMEDY	Outcome	Law	The consequence when an obligation is breached
RESOURCE	Artifact	Comp	A managed asset: file, service, endpoint, or data store
SERVICE	Actor	Comp	A long-lived process that accepts requests
STAGE	Container	Comp	A logical grouping of transforms within a pipeline
STREAM	Artifact	Comp	An ordered, potentially unbounded sequence of data items
TRANSFORM	Actor	Comp	A node that performs exactly one operation

B.2 Verbs (80)

Transform (17)

Word	Sub	Src	Definition
AGGREGATE	Transform	Comp	Reduce collection to single value
APPLY	Transform	Comp	Apply a rule, transform, or patch to a target
BATCH	Transform	Comp	Group items into fixed-size chunks
DERIVE	Transform	Comp	Produce a derived artifact from a source
DISTINCT	Transform	Comp	Remove duplicates
EXCLUDE	Transform	Comp	Remove items satisfying a predicate
FILTER	Transform	Comp	Select items satisfying a predicate
FLATTEN	Transform	Comp	Collapse one level of nesting
GROUP	Transform	Comp	Partition collection by key
MAP	Transform	Comp	Transform each item independently
MERGE	Transform	Comp	Combine multiple collections into one
RENAME	Transform	Comp	Change field names without changing values
SKIP	Transform	Comp	Bypass first N items
SORT	Transform	Comp	Order items by comparison function
SPLIT	Transform	Comp	Divide one collection into multiple
TAKE	Transform	Comp	Return first N items
UPDATE	Transform	Comp	Mutate existing state in place; distinct from WRITE

Move (16)

Word	Sub	Src	Definition
APPEND	Move	Comp	Add an item to the end of an ordered log or list
ASSIGN	Move	Law	Transfer rights or obligations to another party
AUTHENTICATE	Move	Comp	Prove identity to another actor
DELEGATE	Move	Comp	Hand off a stage to another agent to perform
DELIVER	Move	Law	Transfer possession of an artifact to another party
EMIT	Move	Comp	Produce an artifact or event to an outward channel
LOAD	Move	Comp	Read data into working context; distinct from READ
POST	Move	Comp	Publish to a channel or issue thread; distinct from SEND
PRODUCE	Move	Comp	Generate output from an operation
READ	Move	Comp	Input data from external source
RECEIVE	Move	Comp	Accept data from another actor
REQUEST	Move	Comp	Send message expecting a response
RESPOND	Move	Comp	Reply to a prior request
RETURN	Move	Comp	Emit final output and terminate
SEND	Move	Comp	Transmit data to another actor
WRITE	Move	Comp	Output data to external destination

Obligate (6)

Word	Sub	Src	Definition
ASSERT	Obligate	Comp	Declare a condition that must be true
ASSERT_NOT	Obligate	Comp	Declare a condition that must never be true; negative dual of ASSERT
INVARIANT	Obligate	Comp	Declare a condition that must hold across all time, not just at one point
REQUIRE	Obligate	Comp	Declare a hard prerequisite
SHALL	Obligate	Law	Mandatory: party MUST perform action
VALIDATE	Obligate	Comp	Assert data satisfies condition; halt if not

Permit (7)

Word	Sub	Src	Definition
ACCEPT	Permit	Comp	Accept a result or proposal at a decision gate
ALLOW	Permit	Comp	Permit an action without mandating it
APPROVE	Permit	Comp	Accept as satisfying criteria; opposite of REJECT
GRANT	Permit	Law	Confer a right or permission to another party
MAY	Permit	Law	Permitted: party is allowed but not required
OVERRIDE	Permit	Law	Supersede a constraint or prohibition for this action
SUGGEST	Permit	Comp	Recommend a non-binding course of action

Prohibit (4)

Word	Sub	Src	Definition
DENY	Prohibit	Comp	Refuse a requested action
REJECT	Prohibit	Comp	Refuse input that does not meet criteria
REVOKE	Prohibit	Law	Withdraw a previously granted right or permission
SHALL_NOT	Prohibit	Law	Prohibited: party MUST NOT perform action

Control (15)

Word	Sub	Src	Definition
BRANCH	Control	Comp	Route to one of two paths on boolean condition
COMPLETE	Control	Comp	Bring an operation to its finished state
EQUALS	Control	Comp	Evaluates true if left and right values are identical
EXCEEDS	Control	Comp	Evaluates true if left value is greater than right
EXECUTE	Control	Comp	Run a unit of work
EXISTS	Control	Comp	Evaluates true if the noun is present and non-null
EXPIRE	Control	Comp	Transition to EXPIRED state; deadline passed
HALT	Control	Comp	Stop a process immediately
INVOKE	Control	Comp	Call an interface, function, or process
MATCH	Control	Comp	Route to one of N paths on pattern
PASS	Control	Comp	Satisfy a check or gate
PRECEDES	Control	Comp	Evaluates true if left value is less than right
RETRY	Control	Comp	Re-attempt failed operation up to bounded count
THROW	Control	Comp	Generate an error condition; directs flow to error handler
TIMEOUT	Control	Comp	Bound wall-clock time for an operation

Bind (10)

Word	Sub	Src	Definition
ADMINISTER	Bind	Law	Establish and exercise authority over
AUGMENT	Bind	Comp	Add capability to an existing thing
CITE	Bind	Law	Formally refer to another entity
DECLARE	Bind	Comp	State a fact or create a named thing
DEFINE	Bind	Comp	Establish the meaning of a term
HAVE	Bind	Comp	Possess a property or attribute (SHALL HAVE)
IMPLEMENT	Bind	Comp	Fulfill the contract of an interface
NEST	Bind	Comp	Place one thing structurally inside another
REPLACE	Bind	Comp	Entirely substitute one thing for another
STIPULATE	Bind	Law	Establish a binding agreed-upon condition

Resolve (5)

Word	Sub	Src	Definition
CATCH	Resolve	Comp	Intercept an error and enter recovery
CURE	Resolve	Law	Correct a breach within allowed time
HANDLE	Resolve	Comp	Process an error condition
INDEMNIFY	Resolve	Law	Compensate for loss caused by breach
RECOVER	Resolve	Comp	Restore normal operation after failure

B.3 Adjectives (39)

Word	Sub	Src	Definition
ACTIVE	State	Comp	Currently in use or executing
ARRAY	Type	Comp	Ordered collection data type
AT_LEAST	Quantity	Comp	Lower-bound cardinality; the count is N or more
BINDING	State	Law	Creates enforceable obligation
BOOLEAN	Type	Comp	True/false data type
CONFIDENTIAL	Access	Law	Restricted to authorized parties only
CRITICAL	Priority	Comp	Failure is unrecoverable
DEFAULT	Priority	Comp	The value used when none is specified
EMPTY	State	Comp	Present but containing nothing
ENFORCEABLE	State	Law	Can be compelled by remedy
EXACTLY	Quantity	Comp	Exact cardinality; the count is precisely N
EXPIRED	State	Law	Past its deadline; no longer in effect
FAILED	State	Comp	Terminated with error
HIGH	Priority	Comp	Elevated importance
IMMUTABLE	State	Comp	Cannot be changed after creation
INTEGER	Type	Comp	Whole number data type
INVALID	State	Comp	Fails one or more declared constraints
JOINT	Quantity	Law	Shared equally among multiple parties
LOW	Priority	Comp	Reduced importance
MATERIAL	Priority	Law	Significant enough to affect decisions
MULTIPLE	Quantity	Comp	More than one instance may exist
MUTABLE	State	Comp	Can be changed after creation
NULL	State	Comp	Absent; no value
OBJECT	Type	Comp	Key-value structure data type
ONLY	Quantity	Comp	Exclusively; no other instance or case is permitted
OPTIONAL	Quantity	Comp	May be absent; absence is acceptable
PENDING	State	Comp	Created but not yet processed
PRECEDENT	State	Law	Informing but not binding; advisory based on prior decision
PRIVATE	Access	Comp	Accessible only to the owning actor
PROTECTED	Access	Comp	Accessible to the owning actor and its children
PUBLIC	Access	Comp	Accessible to any actor
READONLY	Access	Comp	Can be read but not written
REQUIRED	Quantity	Comp	Must be present; absence is an error
SOLE	Quantity	Law	One and only one; no others permitted
STRING	Type	Comp	Text data type
SUBORDINATE	Priority	Law	Lower in authority hierarchy
UNIQUE	Quantity	Comp	Exactly one instance may exist
VALID	State	Comp	Satisfies all declared constraints
VOID	State	Law	Without legal effect; as if it never existed

B.4 Adverbs (19), Prepositions (18), Conjunctions (13)

Adverbs (19)

Word	Sub	Src	Definition
ALWAYS	Repetition	Comp	Execute every time the condition is met
ASYNC	Timing	Comp	Execute without blocking the caller
BOUNDED	Repetition	Comp	Execute up to N times (requires integer)
CONDITIONALLY	Condition	Law	Only if a stated prerequisite is met
FORTHWITH	Timing	Law	Immediately and without any delay
IMMEDIATE	Timing	Comp	Execute with no delay
LAZY	Timing	Comp	Defer execution until the result is needed
NEVER	Repetition	Comp	Do not execute under any condition
ONCE	Repetition	Comp	Execute exactly one time
PARALLEL	Timing	Comp	Execute steps simultaneously
PARTIALLY	Degree	Comp	Affecting some but not all items
PROMPTLY	Timing	Law	Within a reasonable time, without unnecessary delay
REASONABLY	Degree	Law	As a competent party would under the circumstances
SEQUENTIAL	Timing	Comp	Execute steps one after another in order
STRICTLY	Degree	Comp	With zero tolerance for deviation
SUBSTANTIALLY	Degree	Law	In all material respects; minor deviations tolerated
SYNC	Timing	Comp	Execute and block until complete
UNCONDITIONALLY	Condition	Law	Without any prerequisite or qualification
WITHIN	Timing	Law	Before the specified duration elapses (requires duration)

Prepositions (18)

Word	Sub	Src	Definition
AS	Binding	Comp	Binds an identifier to a type or role
BINDS	Dependency	Comp	Schema constrains what a node accepts or produces
BY	Binding	Comp	Specifies the key, criterion, or basis for an operation
CONTAINS	Containment	Comp	Parent structurally holds child
DEPENDS_ON	Dependency	Comp	Source requires target to function
EXTENDS	Dependency	Comp	Source adds capability to target
FEEDS	Flow	Comp	Data flows unconditionally from source to destination
FROM	Flow	Comp	Directional: data originates at source
GOVERNS	Authority	Law	Source has authority over target
IMPLEMENTS	Dependency	Comp	Source fulfills the contract of target
ON_BEHALF_OF	Authority	Law	Acting with delegated authority for another
PURSUANT_TO	Authority	Law	In accordance with; as directed by
REFERENCES	Reference	Comp	Non-hierarchical cross-reference
RETURNS_TO	Flow	Comp	Output flows back to a previous node
ROUTES	Flow	Comp	Data flows conditionally based on predicate
SUBJECT_TO	Dependency	Law	Source is constrained by target
SUPERSEDES	Replacement	Comp	Source completely replaces target
TO	Flow	Comp	Directional: data moves toward destination

Conjunctions (13)

Word	Sub	Src	Definition
AND	Parallel	Comp	Execute both; both must succeed
ELSE	Alternative	Comp	Execute only when preceding condition is false
EXCEPT	Exception	Law	Everything applies other than the stated carve-out
FINALLY	Sequence	Comp	Execute regardless of success or failure
IF	Cause	Comp	Execute following clause only when condition is true
NOTWITHSTANDING	Exception	Law	Overrides any conflicting clause
OR	Alternative	Comp	Execute alternative if first fails or is false
PROVIDED_THAT	Exception	Law	Clause applies only if the stated condition is met
THEN	Sequence	Comp	Execute next step after current completes
UNLESS	Exception	Law	Clause does not apply if exception condition is met
WHEN	Cause	Comp	Execute following clause upon event occurrence
WHEREAS	Exception	Law	Establishes factual context for what follows (preamble only)
WHILE	Cause	Comp	Continue executing as long as condition holds

B.5 Articles (10), Pronouns (7)

Articles (10)

Word	Sub	Src	Definition
A	Existential	Shared	Exactly one, unspecified which
ALL	Universal	Shared	Every instance without exception
ANY	Existential	Shared	At least one; whichever satisfies
EACH	Universal	Shared	Every instance, considered individually
EVERY	Universal	Shared	Every instance (synonym of ALL, emphasizes completeness)
NO	Negative	Shared	Zero instances
NONE	Negative	Shared	Zero instances (used without following noun)
SOME	Existential	Shared	One or more, unspecified which
THE	Specific	Shared	One specific, previously identified instance
THIS	Specific	Shared	The instance in current scope

Pronouns (7)

Word	Sub	Src	Definition
INPUT	Source	Comp	The data received by the current operation
OUTPUT	Result	Comp	The data produced by a transform
RESULT	Result	Comp	The output of the most recent verb
SAID	Legal reference	Law	The previously named noun (legal "the")
SELF	Self	Comp	The current actor or scope
SOURCE	Source	Comp	The originating node of the current data
TARGET	Target	Comp	The destination node of the current action

B.6 Level Prefixes (21)

The ninth part of speech: hierarchy markers, in magnitude order. Combined with a semantic name they form a `metric_level` identifier (`KILO_REPOSITORY`, `BASE_FUNCTION`); as bare tokens on their own line they are zoom directives that compile to nothing.

Word	Factor	Definition
YOTTA	10^{24}	Largest macro scale
ZETTA	10^{21}	
EXA	10^{18}	
PETA	10^{15}	
TERA	10^{12}	
GIGA	10^{9}	
MEGA	10^{6}	
KILO	10^{3}	
HECTO	10^{2}	
DEKA	10^{1}	Smallest macro scale
BASE	10^{0}	Default consumption level — no prefix scaling, the generally accessible plane
DECI	10^{-1}	Largest micro scale
CENTI	10^{-2}	
MILLI	10^{-3}	
MICRO	10^{-6}	
NANO	10^{-9}	
PICO	10^{-12}	
FEMTO	10^{-15}	
ATTO	10^{-18}	
ZEPTO	10^{-21}	
YOCTO	10^{-24}	Smallest micro scale

B.7 Sugar (open pattern)

Sugar is the pattern `'[a-z_]+`: any apostrophe-prefixed lowercase token. Sugar compiles to nothing—a sentence with sugar and the same sentence without it produce the identical graph. It exists for human readability (`'of`, `'is`, `'that`, `'please`) and for domain annotation (`'postgres`, `'trugs_llc`).

Because it is a pattern rather than a word list, it is unlimited and is not counted in the 233-word vocabulary.

B.8 Summary

Part of Speech	Comp	Law	Shared	Total
Nouns	20	6	—	26
Verbs	67	13	—	80
Adjectives	29	10	—	39
Adverbs	12	7	—	19
Prepositions	14	4	—	18
Conjunctions	8	5	—	13
Articles	—	—	10	10
Pronouns	6	1	—	7
Level prefixes	—	—	21	21
Total	**156**	**46**	**31**	**233**

C Cross-Language Vocabulary

The following table demonstrates that TRUGS vocabulary concepts exist independently across major human languages. These are not translations of English words — they are terms that independently evolved in each language's legal system, computational culture, and grammatical tradition. The universality of these concepts is what makes TRUGS language-agnostic: the grammar operates on parts of speech and composition rules, not on specific words in a specific language.

We present selected vocabulary items across 6 languages: English, Chinese (Mandarin), Spanish, Arabic, Hindi, and French — representing the six most-spoken languages by total speakers and four distinct legal traditions (common law, civil law, Islamic law, Hindu law).

C.1 Modals — Obligation, Permission, Prohibition

These modals are the foundation of governance in every legal system.

TRUGS	English	Chinese	Spanish	Arabic	Hindi	French
SHALL	shall	应当	deberá	yajib	karnaa hogaa	devra
SHALL_NOT	shall not	不得	no deberá	laa yajuuz	nahii karega	ne devra pas
MAY	may	可以	podrá	yajuuz	kar saktaa hai	pourra

C.2 Verbs — Transform Operations

Universal computational operations recognized by programmers in every language.

TRUGS	English	Chinese	Spanish	Arabic	Hindi	French
FILTER	filter	筛选	filtrar	tarsiih	chhaannaa	filtrer
SORT	sort	排序	ordenar	tartiib	kramabadhh	trier
MAP	map	映射	mapear	rasm	pratichitra	mapper
MERGE	merge	合并	fusionar	damj	milaanaa	fusionner
SPLIT	split	拆分	dividir	taqsim	baantnaa	diviser
VALIDATE	validate	验证	validar	tahaqquq	maany karana	valider
READ	read	读取	leer	qiraa'a	parhnaa	lire
WRITE	write	写入	escribir	kitaaba	likhnaa	écrire
SEND	send	发送	enviar	irsaal	bhejna	envoyer
RETRY	retry	重试	reintentar	i'aada	punah prayaas	réessayer

C.3 Nouns — Entities

Actors, artifacts, and outcomes that exist in every computational and legal domain.

TRUGS	English	Chinese	Spanish	Arabic	Hindi	French
PARTY	party	方	parte	taraf	paksh	partie
AGENT	agent	代理	agente	wakiil	abhikartaa	agent
DATA	data	数据	datos	bayaanaat	daataa	données
RECORD	record	记录	registro	sijill	abhilekh	enregistrement
MESSAGE	message	消息	mensaje	risaala	sandesh	message
ERROR	error	错误	error	khata'	truti	erreur
RESULT	result	结果	resultado	natiija	parinaam	résultat
REMEDY	remedy	补救	remedio	ta'diib	upachaar	remède
SERVICE	service	服务	servicio	khidma	sevaa	service

C.4 Adjectives — State and Access Modifiers

TRUGS	English	Chinese	Spanish	Arabic	Hindi	French
VALID	valid	有效	válido	sahiih	maanya	valide
INVALID	invalid	无效	inválido	baatil	amaanya	invalide
ACTIVE	active	活跃	activo	nashiit	sakriya	actif
PENDING	pending	待处理	pendiente	mu'allaq	lambit	en attente
REQUIRED	required	必需	requerido	matluub	aavashyak	requis
OPTIONAL	optional	可选	opcional	ikhtiyaarii	vaikalpik	optionnel
PUBLIC	public	公开	público	'aamm	saarvajanik	public
PRIVATE	private	私有	privado	khaass	nijii	privé
CONFIDENTIAL	confidential	机密	confidencial	sirrii	gopaniiya	confidentiel

C.5 Conjunctions — Flow Control

Computational flow and legal exceptions, present in every language.

TRUGS	English	Chinese	Spanish	Arabic	Hindi	French
THEN	then	然后	entonces	thumma	phir	ensuite
AND	and	和	y	wa	aur	et
OR	or	或	o	aw	yaa	ou
IF	if	如果	si	idhaa	agar	si
UNLESS	unless	除非	a menos que	illaa idhaa	jab tak nahii	sauf si
EXCEPT	except	除外	excepto	illaa	sivay	sauf
NOTWITH-STANDING	notwith-standing	尽管	no obstante	bisarf alnazar	ke baavajud	nonobstant

C.6 Prepositions — Relationships

C.7 Cross-Language Patterns

Several patterns emerge from this analysis:

1. **Legal modals are universal.** Every legal system distinguishes obligation (SHALL/应当/deberá/yajib), prohibition (SHALL_NOT/不得/no deberá/laa yajuuz), and permission (MAY/可以/podrá/yajuuz). Comparative legal scholarship confirms these categories appear across common law, civil law, and Islamic law traditions [Nyazee, 2022], and deontic logic formalizes them as fundamental normative operators [von Wright, 1951]. These are not cultural conventions — they are structural requirements of any governance system.

TRUGS	English	Chinese	Spanish	Arabic	Hindi	French
TO	to	到	a/hacia	ilaa	ko	à/vers
FROM	from	从	de/desde	min	se	de/depuis
GOVERNS	governs	管辖	gobierna	yahkum	shaasit karata	gouverne
SUBJECT_TO	subject to	受制于	sujeto a	khaadi' li	adhiin	soumis à
CONTAINS	contains	包含	contiene	yahtawii	shaamil hai	contient
DEPENDS_ON	depends on	依赖	depende de	ya'tamid 'alaa	nirbhar hai	dépend de

2. **Computational operations are universal.** FILTER, SORT, MAP, MERGE exist in every programming language under the same names or direct equivalents. Chinese programmers use 筛选 (filter) and 排序 (sort) with the same semantics as their English counterparts. The concepts are mathematical, not linguistic.
3. **State modifiers are universal.** VALID/INVALID, PUBLIC/PRIVATE, REQUIRED/OPTIONAL — these binary distinctions exist in every language because they describe states that any system must distinguish. A system that cannot distinguish valid from invalid data cannot validate.
4. **Conjunctions map to control flow universally.** THEN (sequence), AND (parallel), OR (alternative), IF (conditional), UNLESS (exception) — these correspond to computational control structures that exist in every programming paradigm and every legal drafting tradition.
5. **Sugar is language-specific; executable words are not.** Sugar tokens (`'of`, `'is`, `'that`, `'please`) are grammatical particles unique to each language. The executable words (SHALL, FILTER, VALID) are universal concepts with direct equivalents. This validates the TRUGS design: sugar is a rendering layer; the executable vocabulary is structural.

This analysis covers selected vocabulary items across 6 languages. The full 233-word vocabulary can be mapped to any human language whose legal and computational traditions provide equivalent terms — which, as this table demonstrates, is every major language.

D Compilation and Decompilation Algorithms

The following pseudocode describes the compilation (sentence → graph) and decompilation (graph → sentence) algorithms. These algorithms are intentionally straightforward: the complexity resides in the grammar and composition rules, not in the compilation procedure itself.

D.1 Preprocessing: Sugar Stripping

Before parsing, all sugar tokens are removed. This is a single linear pass over the token stream.

```
function strip_sugar(tokens):
    return [t for t in tokens if not SUGAR_PATTERN.match(t.word)]
```

The SUGAR_PATTERN is `'[a-z_]+`: any apostrophe-prefixed lowercase token is sugar. Because sugar carries no semantic content, its removal cannot change the meaning or validity of a sentence. The stripped token stream is the input to the parser.

D.2 Compilation: Sentence to Graph

Compilation proceeds in three phases: tokenization, parsing, and graph emission.

```
function compile(sentence):
    tokens  = tokenize(sentence)          # split on whitespace +
        punctuation
    tokens  = strip_sugar(tokens)         # remove sugar words
    tree    = parse(tokens, BNF_GRAMMAR)  # build parse tree
    validate_composition(tree)            # check rules 10-16
```

```
    graph   = emit_graph(tree)            # walk tree, emit nodes and
        edges
    return graph

function emit_graph(tree):
    graph = new Graph()
    for clause in tree.clauses:
        subject_node = emit_subject(graph, clause.subject)
        op_node      = emit_operation(graph, clause.verb)

        if clause.modal:                  # SHALL, MAY, SHALL_NOT
            graph.add_edge(subject_node, op_node, clause.modal)
        else:
            graph.add_edge(subject_node, op_node, "PERFORMS")

        for adverb in clause.verb.adverbs:
            op_node.properties[adverb.subcategory] = adverb.word
            if adverb.value:
                op_node.properties[adverb.word] = adverb.value

        for obj_phrase in clause.objects:
            if obj_phrase.is_preposition:
                target = emit_noun(graph, obj_phrase.noun)
                graph.add_edge(op_node, target, obj_phrase.preposition
                    )
            elif obj_phrase.is_pronoun:
                antecedent = resolve_pronoun(graph, obj_phrase.pronoun
                    )
                graph.add_edge(op_node, antecedent, "REFERENCES")
            else:
                target = emit_noun(graph, obj_phrase.noun)
                graph.add_edge(op_node, target, "FEEDS")

    for conjunction in tree.conjunctions:
        left  = graph.last_operation_in(conjunction.left_clause)
        right = graph.first_operation_in(conjunction.right_clause)
        graph.add_edge(left, right, conjunction.word)

    return graph

function emit_subject(graph, subject):
    node = new Node(id=subject.identifier, type=subject.noun)
    for adj in subject.adjectives:
        node.properties[adj.subcategory] = adj.word
    if subject.article:
        node.scope = {quantifier: subject.article}
    graph.add_node(node)
    return node

function emit_operation(graph, verb):
    node = new Node(id=auto_id(), type="TRANSFORM")
    node.properties["operation"] = verb.word
    graph.add_node(node)
    return node

function emit_noun(graph, noun_phrase):
    node = new Node(id=noun_phrase.identifier, type=noun_phrase.noun)
    for adj in noun_phrase.adjectives:
        node.properties[adj.subcategory] = adj.word
    if noun_phrase.article:
        node.scope = {quantifier: noun_phrase.article}
    graph.add_node(node)
    return node
```

The key property of this algorithm is that every grammar element maps to exactly one graph-emission call. No heuristics, inference, or probabilistic decisions are involved. The compilation is deterministic: the same sentence always produces the same graph.

D.3 Decompilation: Graph to Sentence

Decompilation reverses the compilation by walking the graph in topological order and emitting words.

```
function decompile(graph):
    sentences = []
    for root in graph.roots():          # nodes with no incoming edges
        sentence = decompile_from(graph, root)
        sentences.append(sentence)
    return join(sentences, ". ") + "."

function decompile_from(graph, node):
    words = []

    # Emit subject
    words += decompile_noun(node)

    # Find outgoing modal/operation edge
    for edge in graph.outgoing(node):
        if edge.relation in MODALS:
            words.append(edge.relation)         # SHALL, MAY,
                SHALL_NOT
            op_node = graph.node(edge.to_id)
            words.append(op_node.properties["operation"])

            # Emit adverbs
            for key, val in op_node.properties.items():
                if key in QUALIFIER_SET:
                    words.append(val)

            # Emit objects
            for obj_edge in graph.outgoing(op_node):
                if obj_edge.relation in PREPOSITION_SET:
                    words.append(obj_edge.relation)
                    words += decompile_noun(graph.node(obj_edge.to_id)
                        )
                elif obj_edge.relation in COMBINATOR_SET:
                    words.append(obj_edge.relation)
                    words += decompile_from(graph, graph.node(obj_edge
                        .to_id))

    return join(words, " ")

function decompile_noun(node):
    words = []
    if node.scope and node.scope.quantifier:
        words.append(node.scope.quantifier)     # ALL, EACH, THE, NO
    for key in [QUANTITY, PRIORITY, STATE, ACCESS, TYPE]:
        if key in node.properties:
            words.append(node.properties[key])  # adjectives in order
    words.append(node.type)                     # RECORD, PARTY, DATA
    if node.id != auto_generated:
        words.append(node.id)                   # identifier
    return words
```

The decompilation algorithm mirrors the compilation algorithm: every graph element maps to exactly one word, and the emission order follows the adjective stacking order (Quantity $>$ Priority $>$ State $>$ Access $>$ Type) defined by the composition rules.

D.4 Round-Trip Verification

Given these algorithms, round-trip verification is mechanical:

```
function verify_round_trip(sentence):
    graph       = compile(sentence)
    recovered   = decompile(graph)
    recompiled  = compile(recovered)
    return graph == recompiled and sentence == recovered
```

The build runs this verification with the released tool on the 30 numbered examples of the specification's example file: 29 compile, and 29 decompile to their source text. The reference implementation of both algorithms is the published `trugs-tools` package (the `trug` CLI); the specification repository is spec-only.

E TRUGS in TRUGS

This appendix presents a tour of the TRUGS Language written in the TRUGS Language: definitions and preambles, obligation and permission, operation chains, the adjective order, every preposition as an edge, every conjunction, and the compile–decompile pair stated in the language itself. It is not the specification—that is the grammar, the composition rules and the vocabulary of Sections 4 and 5 and the language file—but a self-description exercise that the build checks: the released tool compiles the listing and decompiles the compiled graph to the text shown.

The appendix has two parts. Part I is the tour in TRUGS Language—sentences a human reads as English. Part II shows complete graphs printed by the released compiler—JSON a machine reads as structure—for the examples discussed below and in the body. A human will experience them differently. A computational agent will compile both to the same internal representation. That is the point.

Part I — The Language

```
DEFINE "trugs" AS NAMESPACE.
DEFINE "sentence" AS RECORD.
DEFINE "graph" AS DATA.

WHEREAS PARTY language ADMINISTER ALL RECORD sentence AND ALL DATA
    graph.
WHEREAS EACH RECORD sentence REQUIRE NAMESPACE trugs.

EACH PARTY SHALL EXECUTE ALL ACTIVE RECORD sentence.
NO PARTY SHALL EXECUTE INVALID RECORD sentence.
ANY DATA graph EXISTS.

PARTY SHALL VALIDATE.
PARTY MAY APPROVE.
PARTY SHALL_NOT DENY.
PARTY SHALL REQUIRE RECORD sentence.
MODULE CONTAINS PARTY.

PARTY SHALL FILTER DATA
  THEN SORT RESULT
  THEN WRITE RESULT TO ENDPOINT.
PARTY SHALL READ DATA FROM STREAM
  THEN VALIDATE RESULT.
```

```
PARTY MAY RETRY BOUNDED 3 WITHIN 30s
  OR THROW EXCEPTION.
PARTY SHALL CATCH THE EXCEPTION
  THEN HANDLE THE ERROR.
PARTY SHALL DECLARE DATA term.

REQUIRED CRITICAL VALID PUBLIC STRING DATA DEPENDS_ON INTERFACE
    adjective-order.

DATA FEEDS DATA.
DATA ROUTES DATA.
MODULE CONTAINS DATA.
DATA REFERENCES DATA.
DATA SUPERSEDES DATA.
DATA EXTENDS DATA.
DATA BINDS PARTY.
SERVICE IMPLEMENTS INTERFACE.
PARTY GOVERNS PARTY.
PARTY RETURNS_TO PARTY.
PARTY PURSUANT_TO INSTRUMENT.
AGENT ON_BEHALF_OF PARTY.
DATA SUBJECT_TO INTERFACE.
PARTY SHALL SEND DATA TO ENDPOINT.
PARTY SHALL READ DATA FROM STREAM.
PARTY SHALL GROUP DATA BY DATA.

PARTY SHALL FILTER DATA
  THEN SORT RESULT.
PARTY SHALL FILTER DATA
  AND PARTY SHALL SORT DATA.
PARTY SHALL FILTER DATA
  OR REJECT DATA.
IF PARTY THROW EXCEPTION
  THEN PARTY SHALL HANDLE THE ERROR.
IF ACTIVE DATA EXISTS
  THEN PARTY SHALL FILTER DATA
  ELSE PARTY SHALL HALT.
WHEN PARTY SEND MESSAGE TO SERVICE
  THEN SERVICE SHALL VALIDATE THE MESSAGE.
PARTY SHALL FILTER DATA
  UNLESS NO VALID DATA EXISTS.
PARTY SHALL FILTER DATA
  EXCEPT PARTY admin MAY WRITE DATA.
PARTY MAY WRITE DATA
  NOTWITHSTANDING PARTY admin MAY OVERRIDE.
PARTY SHALL WRITE DATA
  PROVIDED_THAT PARTY AUTHENTICATE TO SERVICE.
WHEREAS PARTY GOVERNS ALL DATA.
PARTY SHALL FILTER DATA
  WHILE ACTIVE DATA EXISTS.
PARTY SHALL FILTER DATA
  FINALLY SEND MESSAGE TO PARTY.

PARTY system SHALL VALIDATE ALL ACTIVE RECORD.

PARTY compiler SHALL MAP EACH RECORD sentence TO DATA graph.
PARTY compiler SHALL MAP EACH DATA graph TO RECORD sentence.
THE RECORD sentence EQUALS SAID RECORD sentence.
```

That is the tour: every modal, every conjunction and every preposition of the vocabulary appears in it, in sentences the released tool accepts and decompiles to the text shown. If you can read it, you can read TRUGS.

Part II — The Graph

Every `style=json` listing below is the complete output of `trug trl compile -` from trugs-tools 2.2.0 for its displayed source, supplied on standard input. All 6 graph listings are checked by `trug validate`; 6 validate. Individual-field schemas use `style=jsonfragment` and are labeled as fragments.

E.1 Introductory Filter and Write

```
PARTY system SHALL FILTER ALL ACTIVE RECORD THEN WRITE RESULT TO ENDPOINT
output.
```


```
{
  "name": "trug_graph",
  "version": "1.0.0",
  "type": "PROJECT",
  "dimension": "system",
  "dimensions": {
    "system": {
      "description": "Compiled TRUG graph: trug_graph"
    }
  },
  "nodes": [
    {
      "id": "system",
      "type": "PARTY",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_PARTY",
      "dimension": "system"
    },
    {
      "id": "op-1",
      "type": "TRANSFORM",
      "properties": {
        "operation": "FILTER",
        "verb_subcategory": "transform"
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
    },
    {
      "id": "record-1",
      "type": "RECORD",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_RECORD",
      "dimension": "system"
    },
    {
      "id": "op-2",
      "type": "TRANSFORM",
      "properties": {
        "operation": "WRITE",
        "verb_subcategory": "move"
      },
```

```
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
    },
    {
      "id": "output",
      "type": "ENDPOINT",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_ENDPOINT",
      "dimension": "system"
    },
    {
      "id": "root",
      "type": "PROJECT",
      "properties": {},
      "parent_id": null,
      "contains": [
        "system",
        "op-1",
        "record-1",
        "op-2",
        "output"
      ],
      "metric_level": "BASE_PROJECT",
      "dimension": "system"
    }
  ],
  "edges": [
    {
      "from_id": "system",
      "to_id": "op-1",
      "relation": "SHALL",
      "properties": {
        "np_shape": {
          "noun_type": "PARTY"
        }
      }
    },
    {
      "from_id": "op-1",
      "to_id": "record-1",
      "relation": "ACTS_ON",
      "properties": {
        "np_shape": {
          "noun_type": "RECORD",
          "article": "ALL",
          "state": "ACTIVE"
        }
      }
    },
    {
      "from_id": "system",
      "to_id": "op-2",
      "relation": "EXECUTES",
      "properties": {
        "np_shape": {
          "noun_type": "PARTY"
        }
      }
    },
    {
      "from_id": "op-2",
```

```
      "to_id": "op-1",
      "relation": "ACTS_ON",
      "properties": {
        "pronoun": "RESULT"
      }
    },
    {
      "from_id": "op-2",
      "to_id": "output",
      "relation": "TO",
      "properties": {
        "np_shape": {
          "noun_type": "ENDPOINT"
        }
      }
    },
    {
      "from_id": "op-1",
      "to_id": "op-2",
      "relation": "THEN"
    }
  ],
  "capabilities": {
    "vocabularies": [
      "core_v2.0.0"
    ]
  }
}
```


E.2 Validation Example

PARTY system SHALL VALIDATE ALL PENDING RECORD.


```
{
  "name": "trug_graph",
  "version": "1.0.0",
  "type": "PROJECT",
  "dimension": "system",
  "dimensions": {
    "system": {
      "description": "Compiled TRUG graph: trug_graph"
    }
  },
  "nodes": [
    {
      "id": "system",
      "type": "PARTY",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_PARTY",
      "dimension": "system"
    },
    {
      "id": "op-1",
      "type": "TRANSFORM",
      "properties": {
        "operation": "VALIDATE",
        "verb_subcategory": "obligate"
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
```

```
    },
    {
      "id": "record-1",
      "type": "RECORD",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_RECORD",
      "dimension": "system"
    },
    {
      "id": "root",
      "type": "PROJECT",
      "properties": {},
      "parent_id": null,
      "contains": [
        "system",
        "op-1",
        "record-1"
      ],
      "metric_level": "BASE_PROJECT",
      "dimension": "system"
    }
  ],
  "edges": [
    {
      "from_id": "system",
      "to_id": "op-1",
      "relation": "SHALL",
      "properties": {
        "np_shape": {
          "noun_type": "PARTY"
        }
      }
    },
    {
      "from_id": "op-1",
      "to_id": "record-1",
      "relation": "ACTS_ON",
      "properties": {
        "np_shape": {
          "noun_type": "RECORD",
          "article": "ALL",
          "state": "PENDING"
        }
      }
    }
  ],
  "capabilities": {
    "vocabularies": [
      "core_v2.0.0"
    ]
  }
}
```

Foundation (DEFINE + WHEREAS)

```
DEFINE "trugs" AS NAMESPACE.
DEFINE "sentence" AS RECORD.
DEFINE "graph" AS DATA.

WHEREAS PARTY language ADMINISTER ALL RECORD sentence AND ALL DATA
    graph.
WHEREAS EACH RECORD sentence REQUIRE NAMESPACE trugs.
```

```
{
  "name": "trug_graph",
  "version": "1.0.0",
  "type": "PROJECT",
  "dimension": "system",
  "dimensions": {
    "system": {
      "description": "Compiled TRUG graph: trug_graph"
    }
  },
  "nodes": [
    {
      "id": "trugs",
      "type": "NAMESPACE",
      "properties": {
        "defined": true,
        "name": "trugs"
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_NAMESPACE",
      "dimension": "system"
    },
    {
      "id": "sentence",
      "type": "RECORD",
      "properties": {
        "defined": true,
        "name": "sentence"
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_RECORD",
      "dimension": "system"
    },
    {
      "id": "graph",
      "type": "DATA",
      "properties": {
        "defined": true,
        "name": "graph"
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_DATA",
      "dimension": "system"
    },
    {
      "id": "language",
      "type": "PARTY",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_PARTY",
      "dimension": "system"
    },
    {
      "id": "op-1",
      "type": "TRANSFORM",
      "properties": {
        "operation": "ADMINISTER",
        "verb_subcategory": "bind",
        "preamble": true,
```

```
        "preceded_by_blank_line": true
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
    },
    {
      "id": "op-2",
      "type": "TRANSFORM",
      "properties": {
        "operation": "REQUIRE",
        "verb_subcategory": "obligate",
        "preamble": true
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
    },
    {
      "id": "root",
      "type": "PROJECT",
      "properties": {},
      "parent_id": null,
      "contains": [
        "trugs",
        "sentence",
        "graph",
        "language",
        "op-1",
        "op-2"
      ],
      "metric_level": "BASE_PROJECT",
      "dimension": "system"
    }
  ],
  "edges": [
    {
      "from_id": "language",
      "to_id": "op-1",
      "relation": "EXECUTES",
      "properties": {
        "np_shape": {
          "noun_type": "PARTY"
        }
      }
    },
    {
      "from_id": "op-1",
      "to_id": "sentence",
      "relation": "ACTS_ON",
      "properties": {
        "np_shape": {
          "noun_type": "RECORD",
          "article": "ALL"
        },
        "chain_id": 1
      }
    },
    {
      "from_id": "op-1",
      "to_id": "graph",
      "relation": "ACTS_ON",
      "properties": {
```

```
        "np_shape": {
          "noun_type": "DATA",
          "article": "ALL"
        },
        "chain_id": 1
      }
    },
    {
      "from_id": "sentence",
      "to_id": "op-2",
      "relation": "EXECUTES",
      "properties": {
        "np_shape": {
          "noun_type": "RECORD",
          "article": "EACH"
        }
      }
    },
    {
      "from_id": "op-2",
      "to_id": "trugs",
      "relation": "ACTS_ON",
      "properties": {
        "np_shape": {
          "noun_type": "NAMESPACE"
        }
      }
    }
  ],
  "capabilities": {
    "vocabularies": [
      "core_v2.0.0"
    ]
  }
}
```


The `DEFINE` sentences create named nodes with definition properties. The `WHEREAS` clauses are marked by `preamble` properties on their operation nodes. The compiler places these nodes under the generated project root. The `ACTS_ON` edges and their properties retain the object phrases.

Operation Chain (SHALL + THEN + TO)

```
PARTY system SHALL FILTER ALL ACTIVE RECORD THEN SORT RESULT THEN WRITE
RESULT TO ENDPOINT output.
```


```
{
  "name": "trug_graph",
  "version": "1.0.0",
  "type": "PROJECT",
  "dimension": "system",
  "dimensions": {
    "system": {
      "description": "Compiled TRUG graph: trug_graph"
    }
  },
  "nodes": [
    {
      "id": "system",
      "type": "PARTY",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_PARTY",
      "dimension": "system"
```

```
    },
    {
      "id": "op-1",
      "type": "TRANSFORM",
      "properties": {
        "operation": "FILTER",
        "verb_subcategory": "transform"
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
    },
    {
      "id": "record-1",
      "type": "RECORD",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_RECORD",
      "dimension": "system"
    },
    {
      "id": "op-2",
      "type": "TRANSFORM",
      "properties": {
        "operation": "SORT",
        "verb_subcategory": "transform"
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
    },
    {
      "id": "op-3",
      "type": "TRANSFORM",
      "properties": {
        "operation": "WRITE",
        "verb_subcategory": "move"
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
    },
    {
      "id": "output",
      "type": "ENDPOINT",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_ENDPOINT",
      "dimension": "system"
    },
    {
      "id": "root",
      "type": "PROJECT",
      "properties": {},
      "parent_id": null,
      "contains": [
        "system",
        "op-1",
        "record-1",
        "op-2",
```

```
        "op-3",
        "output"
      ],
      "metric_level": "BASE_PROJECT",
      "dimension": "system"
    }
  ],
  "edges": [
    {
      "from_id": "system",
      "to_id": "op-1",
      "relation": "SHALL",
      "properties": {
        "np_shape": {
          "noun_type": "PARTY"
        }
      }
    },
    {
      "from_id": "op-1",
      "to_id": "record-1",
      "relation": "ACTS_ON",
      "properties": {
        "np_shape": {
          "noun_type": "RECORD",
          "article": "ALL",
          "state": "ACTIVE"
        }
      }
    },
    {
      "from_id": "system",
      "to_id": "op-2",
      "relation": "EXECUTES",
      "properties": {
        "np_shape": {
          "noun_type": "PARTY"
        }
      }
    },
    {
      "from_id": "op-2",
      "to_id": "op-1",
      "relation": "ACTS_ON",
      "properties": {
        "pronoun": "RESULT"
      }
    },
    {
      "from_id": "system",
      "to_id": "op-3",
      "relation": "EXECUTES",
      "properties": {
        "np_shape": {
          "noun_type": "PARTY"
        }
      }
    },
    {
      "from_id": "op-3",
      "to_id": "op-2",
      "relation": "ACTS_ON",
      "properties": {
        "pronoun": "RESULT"
      }
```

```
    },
    {
      "from_id": "op-3",
      "to_id": "output",
      "relation": "TO",
      "properties": {
        "np_shape": {
          "noun_type": "ENDPOINT"
        }
      }
    },
    {
      "from_id": "op-1",
      "to_id": "op-2",
      "relation": "THEN"
    },
    {
      "from_id": "op-2",
      "to_id": "op-3",
      "relation": "THEN"
    }
  ],
  "capabilities": {
    "vocabularies": [
      "core_v2.0.0"
    ]
  }
}
```


SHALL links the actor to the initial operation; EXECUTES links it to the subsequent operations. ACTS_ON edges record the objects and RESULT references. The object edge's np_shape records ALL ACTIVE. The THEN edges record the order FILTER → SORT → WRITE, and TO links the final operation to the endpoint.

Exception with Authentication (EXCEPT + PROVIDED_THAT)

NO PARTY SHALL WRITE CONFIDENTIAL RESOURCE EXCEPT PARTY owner MAY WRITE CONFIDENTIAL RESOURCE PROVIDED_THAT PARTY owner AUTHENTICATE TO SERVICE auth.


```
{
  "name": "trug_graph",
  "version": "1.0.0",
  "type": "PROJECT",
  "dimension": "system",
  "dimensions": {
    "system": {
      "description": "Compiled TRUG graph: trug_graph"
    }
  },
  "nodes": [
    {
      "id": "party-1",
      "type": "PARTY",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_PARTY",
      "dimension": "system"
    },
    {
      "id": "op-1",
      "type": "TRANSFORM",
      "properties": {
```

```
        "operation": "WRITE",
        "verb_subcategory": "move"
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
    },
    {
      "id": "resource-1",
      "type": "RESOURCE",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_RESOURCE",
      "dimension": "system"
    },
    {
      "id": "owner",
      "type": "PARTY",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_PARTY",
      "dimension": "system"
    },
    {
      "id": "op-2",
      "type": "TRANSFORM",
      "properties": {
        "operation": "WRITE",
        "verb_subcategory": "move",
        "depth": 1
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
    },
    {
      "id": "resource-2",
      "type": "RESOURCE",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_RESOURCE",
      "dimension": "system"
    },
    {
      "id": "op-3",
      "type": "TRANSFORM",
      "properties": {
        "operation": "AUTHENTICATE",
        "verb_subcategory": "move",
        "depth": 2
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
    },
    {
      "id": "auth",
      "type": "SERVICE",
      "properties": {},
```

```
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_SERVICE",
      "dimension": "system"
    },
    {
      "id": "root",
      "type": "PROJECT",
      "properties": {},
      "parent_id": null,
      "contains": [
        "party-1",
        "op-1",
        "resource-1",
        "owner",
        "op-2",
        "resource-2",
        "op-3",
        "auth"
      ],
      "metric_level": "BASE_PROJECT",
      "dimension": "system"
    }
  ],
  "edges": [
    {
      "from_id": "party-1",
      "to_id": "op-1",
      "relation": "SHALL",
      "properties": {
        "np_shape": {
          "noun_type": "PARTY",
          "article": "NO"
        }
      }
    },
    {
      "from_id": "op-1",
      "to_id": "resource-1",
      "relation": "ACTS_ON",
      "properties": {
        "np_shape": {
          "noun_type": "RESOURCE",
          "access": "CONFIDENTIAL"
        }
      }
    },
    {
      "from_id": "owner",
      "to_id": "op-2",
      "relation": "MAY",
      "properties": {
        "np_shape": {
          "noun_type": "PARTY"
        }
      }
    },
    {
      "from_id": "op-2",
      "to_id": "resource-2",
      "relation": "ACTS_ON",
      "properties": {
        "np_shape": {
          "noun_type": "RESOURCE",
          "access": "CONFIDENTIAL"
```

```
        }
      }
    },
    {
      "from_id": "owner",
      "to_id": "op-3",
      "relation": "EXECUTES",
      "properties": {
        "np_shape": {
          "noun_type": "PARTY"
        }
      }
    },
    {
      "from_id": "op-3",
      "to_id": "auth",
      "relation": "TO",
      "properties": {
        "np_shape": {
          "noun_type": "SERVICE"
        }
      }
    },
    {
      "from_id": "op-1",
      "to_id": "op-2",
      "relation": "EXCEPT"
    },
    {
      "from_id": "op-2",
      "to_id": "op-3",
      "relation": "PROVIDED_THAT"
    }
  ],
  "capabilities": {
    "vocabularies": [
      "core_v2.0.0"
    ]
  }
}
```

The initial `SHALL` edge retains the subject's `NO` article in its noun-phrase properties. The `EXCEPT` and `PROVIDED_THAT` edges connect the nested clauses; the nested operation nodes carry their depth.

Sugar Equivalence

`PARTY system SHALL 'please VALIDATE ALL 'of 'the ACTIVE RECORD.`

and

`PARTY system SHALL VALIDATE ALL ACTIVE RECORD.`

With both sentences supplied on standard input, the released compiler prints the following identical graph:

```
{
  "name": "trug_graph",
  "version": "1.0.0",
  "type": "PROJECT",
  "dimension": "system",
  "dimensions": {
    "system": {
      "description": "Compiled TRUG graph: trug_graph"
    }
  },
```

```
  "nodes": [
    {
      "id": "system",
      "type": "PARTY",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_PARTY",
      "dimension": "system"
    },
    {
      "id": "op-1",
      "type": "TRANSFORM",
      "properties": {
        "operation": "VALIDATE",
        "verb_subcategory": "obligate"
      },
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_TRANSFORM",
      "dimension": "system"
    },
    {
      "id": "record-1",
      "type": "RECORD",
      "properties": {},
      "parent_id": "root",
      "contains": [],
      "metric_level": "DECI_RECORD",
      "dimension": "system"
    },
    {
      "id": "root",
      "type": "PROJECT",
      "properties": {},
      "parent_id": null,
      "contains": [
        "system",
        "op-1",
        "record-1"
      ],
      "metric_level": "BASE_PROJECT",
      "dimension": "system"
    }
  ],
  "edges": [
    {
      "from_id": "system",
      "to_id": "op-1",
      "relation": "SHALL",
      "properties": {
        "np_shape": {
          "noun_type": "PARTY"
        }
      }
    },
    {
      "from_id": "op-1",
      "to_id": "record-1",
      "relation": "ACTS_ON",
      "properties": {
        "np_shape": {
          "noun_type": "RECORD",
          "article": "ALL",
          "state": "ACTIVE"
```

```
        }
      }
    }
  ],
  "capabilities": {
    "vocabularies": [
      "core_v2.0.0"
    ]
  }
}
```

`'please`, `'of` and `'the` stripped. Identical graph. Sugar is a human rendering layer.

The Seven Node Fields

The following field-schema fragment shows the 7 required fields of every node:

```
{
  "id": "unique_identifier",
  "type": "ENTITY_TYPE",
  "properties": {"key": "value"},
  "parent_id": "parent_id_or_null",
  "contains": ["child_id_1", "child_id_2"],
  "metric_level": "PREFIX_NAME",
  "dimension": "declared_dimension"
}
```

The following field-schema fragment shows the 3 required fields of every edge:

```
{
  "from_id": "source_node",
  "to_id": "target_node",
  "relation": "RELATION_TYPE"
}
```

Part I reads like English. Part II reads like JSON. A human experiences them as two different documents. A computational agent compiles both to the same internal representation — the same nodes, the same edges, the same topology. That is the point of TRUGS: two formats, one artifact, zero translation.

If you have read both parts, you understand both TRUGS formats. You can write a sentence and know its graph. You can read a graph and know its sentence. The language describes itself in itself. TRUGs all the way down.

F TRUGS as Code Comments

A natural consequence of a language that compiles to executable graphs is that it can serve as inline documentation that is simultaneously human-readable and machine-compilable. We propose a convention: TRUGS Language sentences embedded in source code comments, delimited by `<tr>` tags.

F.1 Block Comments as Contracts

A function-level comment specifies the contract the implementation must satisfy:

```
# <tr>
# PARTY validator SHALL VALIDATE ALL REQUIRED RECORD
#   SUBJECT_TO INTERFACE schema.
# IF PARTY validator THROW EXCEPTION
#   THEN SEND ERROR TO PARTY caller.
# NO PARTY SHALL WRITE INVALID RECORD.
# </tr>
def validate_records(records, schema):
    results = []
    for record in records:
        if not schema.validate(record):
            raise ValidationError(f"Invalid: {record.id}")
        results.append(record)
    return results
```

The <tr> block is not a description of what the code does. It is a **compilable specification** of what the code must do. The block specifies validation of all required records against a schema, error notification on failure, and a prohibition on writing invalid records. Compiling the block produces one graph containing these clauses.

F.2 Inline Comments as Specifications

Individual lines of code can carry inline TRUGS annotations:

```
x = filter(records, active=True)    # <tr>PARTY system SHALL FILTER ALL ACTIVE RECORD.</tr
    >
y = sorted(x, key=priority)         # <tr>PARTY system SHALL SORT RECORD x.</tr>
send(y, endpoint)                   # <tr>PARTY system SHALL WRITE RECORD y TO ENDPOINT
    output.</tr>

if not authenticate(user):         # <tr>IF NO PARTY AUTHENTICATE
    #   THEN PARTY system SHALL THROW EXCEPTION.</tr>
    raise PermissionDenied()       # <tr>PARTY system SHALL THROW EXCEPTION.</tr>
```

Each tagged annotation compiles independently to a graph and validates under the CORE rules. Here `system` names the actor, `x` and `y` name the values on the corresponding code lines, and `output` names the endpoint. The annotations specify filtering, sorting and writing, followed by an authentication condition and an exception. The annotation on the `if` line spans two comment lines but is one complete sentence; the annotation on the `raise` line is independently complete. For comparison, the following sentence connects FILTER, SORT and WRITE with explicit THEN clauses and includes an authentication condition:

```
PARTY system SHALL FILTER ALL ACTIVE RECORD
  THEN SORT RESULT
  THEN WRITE RESULT TO ENDPOINT
  IF NO PARTY AUTHENTICATE
  THEN THROW EXCEPTION.
```

The independent annotations and the block sentence contain FILTER, SORT, WRITE, AUTHENTICATE and THROW operations. Their graphs differ: separate sentences do not introduce the block sentence's THEN links, and the inline annotations name the code values explicitly.

F.3 The Shadow Graph

We call the graphs compiled from tagged source comments a **shadow graph** of the annotated contracts. Its coverage depends on what the comments specify.

```
# <tr>MODULE api CONTAINS FUNCTION auth
#   AND FUNCTION handle AND FUNCTION respond.</tr>

# <tr>PARTY user SHALL REQUEST PARTY api.</tr>
def handle(request):
    user = authenticate(request)    # <tr>PARTY system SHALL AUTHENTICATE TO SERVICE auth
        .</tr>
    data = query(user)              # <tr>PARTY system SHALL READ DATA FROM STREAM db.</tr
        >
```

```
    return format(data)            # <tr>PARTY system SHALL WRITE DATA response TO PARTY
        user.</tr>
```

Here `response` names the formatted return value. For these listings, extraction takes the text between each `<tr>` and `</tr>` pair, removes the host language's comment leaders on continuation lines and surrounding whitespace, and supplies a terminating newline. The paper's checker compiles each extracted annotation separately, validates its graph and compares the decompiled text byte for byte. Of 14 tagged annotations, 14 compile, 14 validate and 9 round-trip identically. The remaining annotations differ from their canonical decompilation. In particular, the coordinated module annotation decompiles to `MODULE api CONTAINS FUNCTION auth.`, its first contained function only. Extraction alone does not connect the separate graphs or establish coverage of unannotated code. The resulting artifacts are:

- **Human-authored** — developers write the comments, not an AI inference engine.
- **Formally specified** — every tagged annotation shown here compiles against the same grammar and composition rules as the paper's other TRUGS sentences.
- **Mechanically validatable** — the released validator reports VALID for each extracted graph under its CORE ruleset.
- **Located with the code** — comments can be maintained alongside the code, though validation of the comments does not establish that the implementation satisfies them.

F.4 Implications

This convention inverts the traditional relationship between code and documentation. In conventional practice, documentation describes code—and the description drifts from reality because no mechanical link enforces consistency. With TRUGS comments, the documentation *is* a compilable artifact that can be validated independently of the code. Checking that a graph is valid does not check the host-language implementation against its contract; that requires a separate comparison of the code and the specified obligations.

F.5 Cross-Language Portability

A further consequence: if the shadow graph is the program, then the source code is merely one rendering of it—just as the TRUGS Language sentence and the JSON graph are two renderings of the same artifact. A computational agent that reads the `<tr>` comments can emit the same graph as code in any target language.

The same graph in Python:

```python
# <tr>PARTY system SHALL FILTER ALL ACTIVE RECORD
#    THEN SORT RESULT THEN WRITE RESULT TO ENDPOINT output.</tr>
x = [r for r in records if r.active]
x.sort(key=lambda r: r.priority)
send(x, endpoint)
```

The same graph in Go:

```go
// <tr>PARTY system SHALL FILTER ALL ACTIVE RECORD
//    THEN SORT RESULT THEN WRITE RESULT TO ENDPOINT output.</tr>
filtered := Filter(records, func(r Record) bool { return r.Active })
sort.Slice(filtered, func(i, j int) bool {
    return filtered[i].Priority < filtered[j].Priority
})
Send(filtered, endpoint)
```

The same graph in Rust:

```rust
// <tr>PARTY system SHALL FILTER ALL ACTIVE RECORD
//    THEN SORT RESULT THEN WRITE RESULT TO ENDPOINT output.</tr>
let mut filtered: Vec<_> = records.iter()
```

```
    .filter(|r| r.active).collect();
filtered.sort_by_key(|r| r.priority);
send(&filtered, &endpoint);
```

The <tr> comment is identical in all three files. The code below it differs—each language has its own syntax, idioms, and type system. But the graph is the same: FILTER → SORT → WRITE, with ACTIVE as a state modifier and ALL as a universal selector.

A computational agent performing language translation does not translate Python to Rust. It reads the shadow graph from the <tr> comments, discards the source language implementation, and emits a new implementation in the target language—guided by the same graph that guided the original author. The graph is the source of truth. The code in any particular language is a rendering.

This is code portability without transpilation. The TRUGS graph is the universal intermediate representation that every language renders differently but every computational agent reads identically.

Comments become contracts. The codebase becomes a graph. The graph transcends language. TRUGs all the way down.